

Principles of Distributed Systems

第三周：流程与线

Week 3: Processes and Threads

By: Fahad Sabah

计算机学院,
北京工业大学, 中国

College of Computer Science,
Beijing University of Technology, China

fahad.sabah@bjut.edu.cn



Recap of Week2

1. Architectural Styles

2. System Architectures

3. Middleware

4. Self-Management

5. Practical: simple client-server architecture and socket programming

第二周回顾

1. 建筑风格

2. 系统架构

3. 中间件

4. 自我管理

5. 实用：简单的客户端-服务器架构和套接字编程

Contents

目录

1. Processes & Threads Basics

1. 流程与线程基础

2. Multithreading in Distributed Systems

2. 分布式系统中的多线程

3. Virtualization (VMs, Containers)

3. 虚拟化 (虚拟机、容器)

4. Multi-threaded client-server + virtualization setup

4. 多线程客户端-服务器+虚拟化设置

5. Project Discussion

5. 项目讨论

Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

进程与线程 (基础)

Processes and Threads (Basics)

Learning objectives

- ☐ understand what a process is in distributed systems
- ☐ explain the difference between process vs thread
- ☐ understand multithreading and why it is needed
- ☐ understand virtualization (VMs, containers) build a multi-threaded client-server application
- ☐ understand basic setup using VMs

学习目标

- ☐ 了解分布式系统中的进程是什么
- ☐ 请解释进程和线程的区别
- ☐ 理解多线程及其必要性
- ☐ 理解虚拟化（虚拟机、容器）构建多线程客户端-服务器应用
- ☐ 理解使用虚拟机的基本设置

热身

Warm-up

When you open:

- ☐ a browser
- ☐ WeChat
- ☐ a music app



👉 Are these one program or many running at the same time?

- ☐ many programs run together
- ☐ Each running program is called a process
- ☐ Inside each process, there can be multiple threads working together

当你打开时:

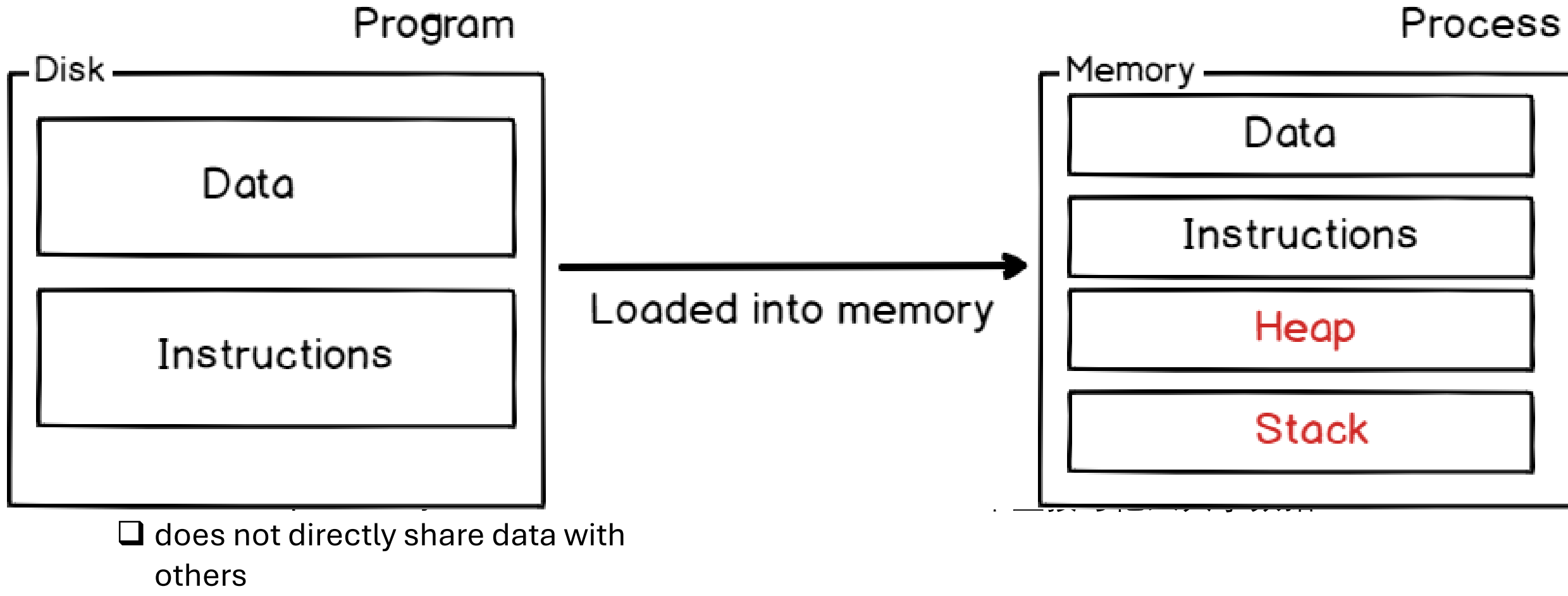
- ☐ 一个浏览器
- ☐ 微信
- ☐ 一款音乐应用

👉 这些程序是同时运行一个程序还是多个程序同时运行?

- ☐ 许多项目同时运行
- ☐ 每个运行中的程序称为进程
- ☐ 每个进程内部可以有多个线程协同工作

什么是建筑?

What is a Process?



什么是线索?

What is a Thread?

Simple definition

A thread is a small unit of execution within a process.

👉 Thread = a worker

💡 Key idea

one process can have many threads share:

- ☐ Memory
- ☐ resources

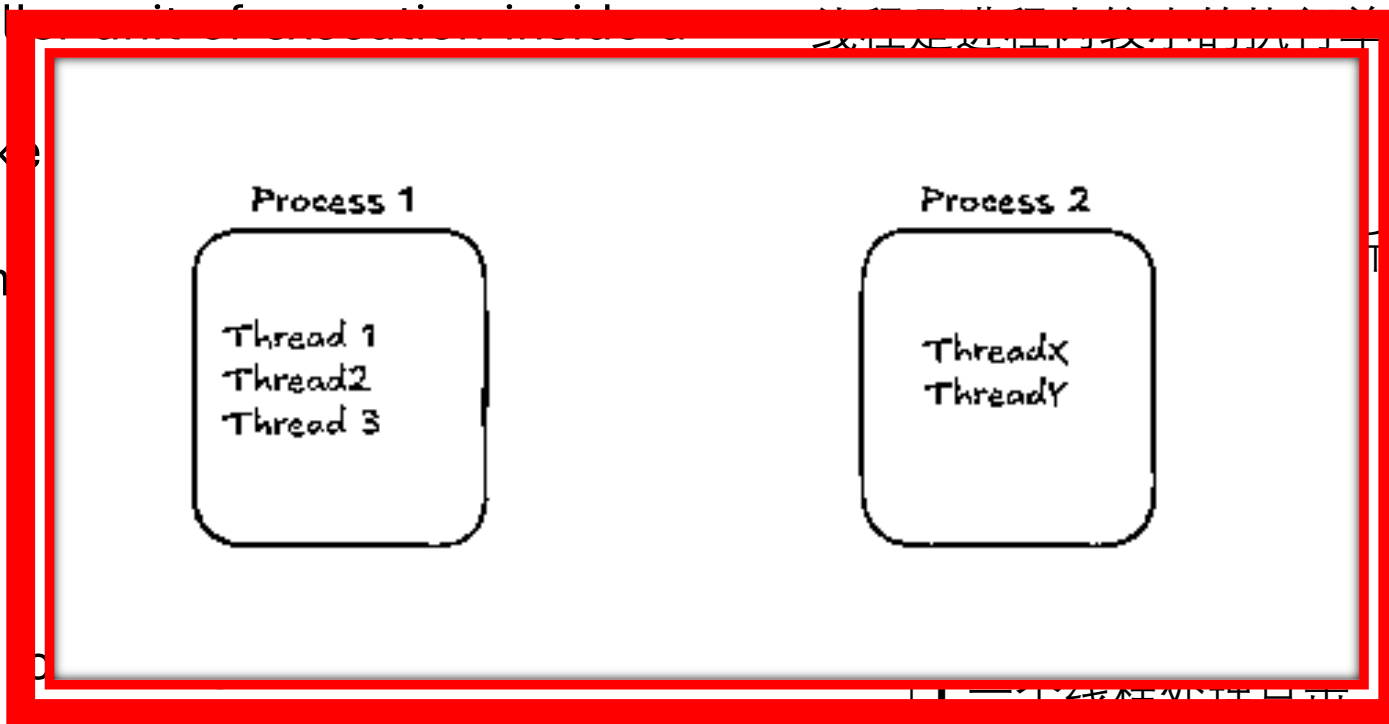
Example

In a browser:

- ☐ one thread handles clicks
- ☐ one thread renders clicks
- ☐ one thread loads images

简单定义

线程是进程内较小的执行单元。



所有帖子分享:

- ☐ 一个线程处理点击
- ☐ 一个线程加载图像

简单比喻（非常重要）

Simple Analogy (Very Important)

Restaurant analogy

Concept	Real-world
Process	Restaurant
Thread	Workers
Task	Serving customers

Explanation

One restaurant (process)

Many workers (threads)

Workers cooperate to serve customers

👉 More workers = faster service

A process is like a company, and threads are the employees.

餐厅类比

解释

一家餐厅（流程）

多工人（线程）

员工合作服务客户

👉 更多工人=更快的服务

流程就像一家公司，线程是员工。

进程与线程 (核心比较)

Process vs Thread (Core Comparison)

Feature	Process	Thread
Meaning	Program running	Task inside program
Memory	Separate	Shared
Speed	Slower	Faster
Creation cost	High	Low
Communication	Hard	Easy

- ☐ Processes are independent
- ☐ Threads are cooperative

- ☐ 进程是独立的
- ☐ 线程是合作式的

In Simple Words

Processes are like separate houses, threads are like people inside the same house.

简单来说

进程就像两个独立的房子，线程就像同一个房子里的人。

为什么线程很重要？

Why Threads are Important?

Without threads

Server handles:
one client at a time

👉 Very slow

With threads

Server can:
handle many clients
at the same time

👉 Faster and scalable

没有线

服务器账号:
一次只负责一个客户

👉 非常慢

带线

服务器可以:
处理大量客户
同时

👉 更快且可扩展

Threads allow systems to do many things at once.

线程允许系统同时执行多种任务。

现实世界的例子

Real-world Examples

Example 1: WeChat

- one thread → send messages
- one thread → receive messages
- one thread → notifications

Example 2: Web Server

- many users request pages
- server creates threads
- each thread handles one user

Example 3: E-commerce

- one thread → search
- one thread → payment
- one thread → notifications

例子1：微信

- 一个线程→发送消息
- 一个线程→接收消息
- 一个提示→线程

示例2：网页服务器

- 许多用户请求页面
- 服务器创建线程
- 每个线程处理一个用户

例子3：电子商务

- 一个搜索→线索
- 一→付款线索
- 一个提示→线程

多任务处理与多线程

Multitasking vs Multithreading

Multitasking

many processes running

example: Chrome + WeChat + Music

多任务处理

许多进程正在运行

示例: Chrome + 微信 + 音乐

Multithreading

one process with many threads

多线程

一个多线程进程

👉 Multitasking = many apps

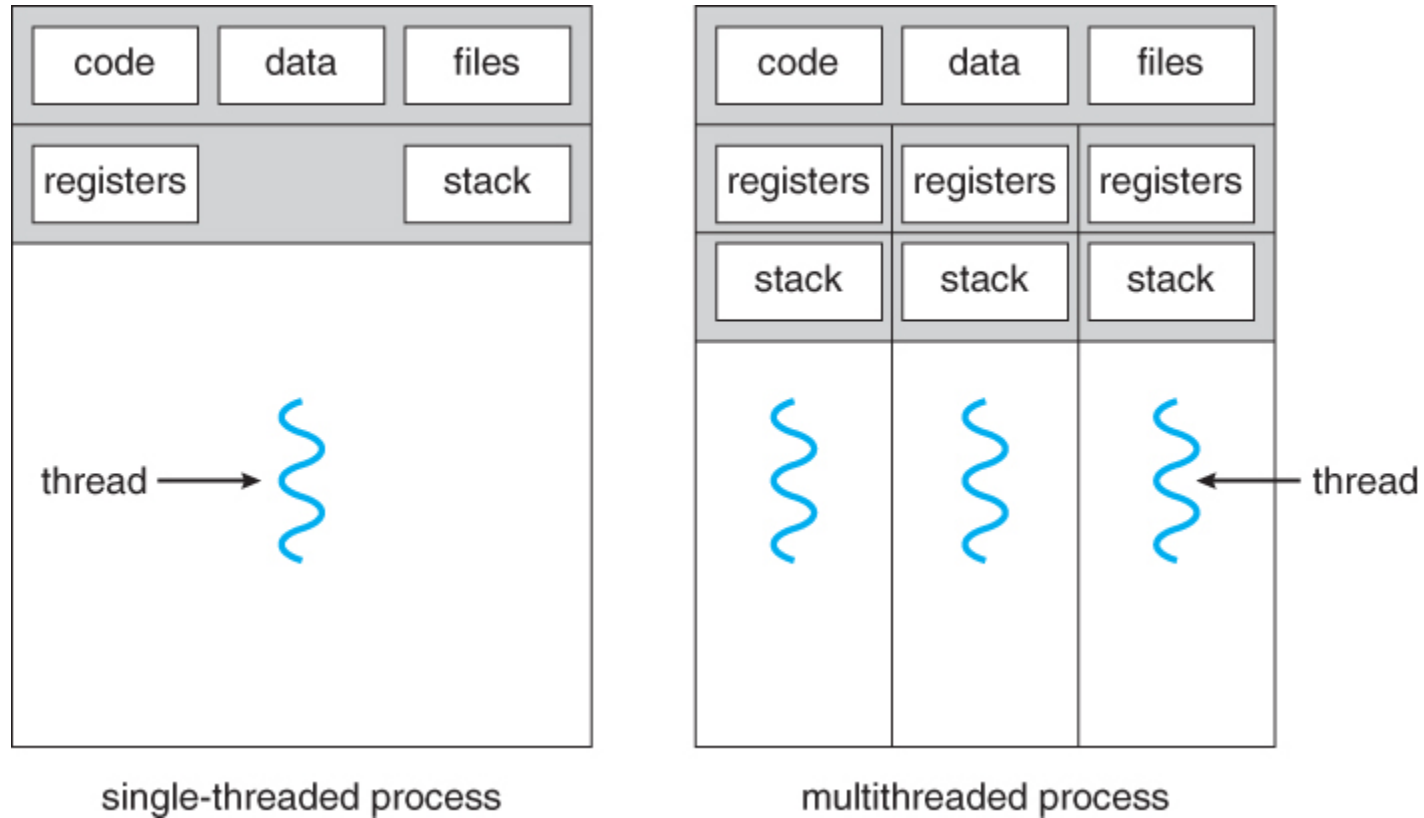
👉 Multithreading = many tasks inside one
app

👉 多任务处理 = 多个应用

👉 多线程 = 一个应用内的多个任务

视觉理解

Visual Understanding



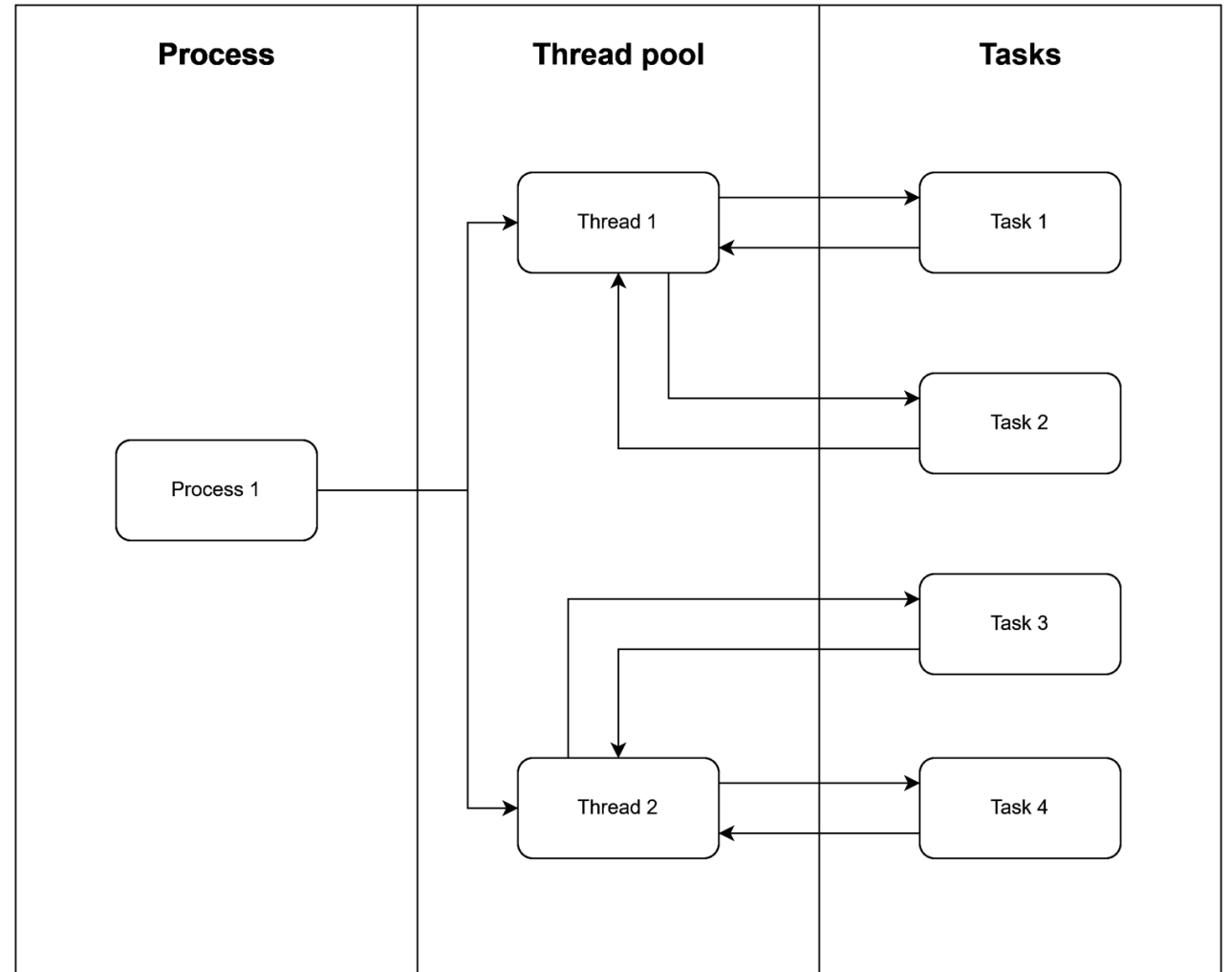
基础线程工作流程

Basic Thread Workflow

Steps:

1. process starts
2. threads are created
3. each thread performs a task
4. threads may run simultaneously

Threads divide work inside a program.



简洁的建筑风格

Problems with Threads

1. Shared memory problem

Threads share data → can conflict

2. Race condition

Two threads try to update same data

3. Synchronization needed

Need rules to coordinate threads

1. 共享内存问题

线程共享数据→可能冲突

2. 竞赛条件

两个线程尝试更新相同的数据

3. 需要同步

需要规则来协调线程

Threads make systems faster, but also more complex.

线程让系统更快，但也更复杂。

简洁的建筑风格

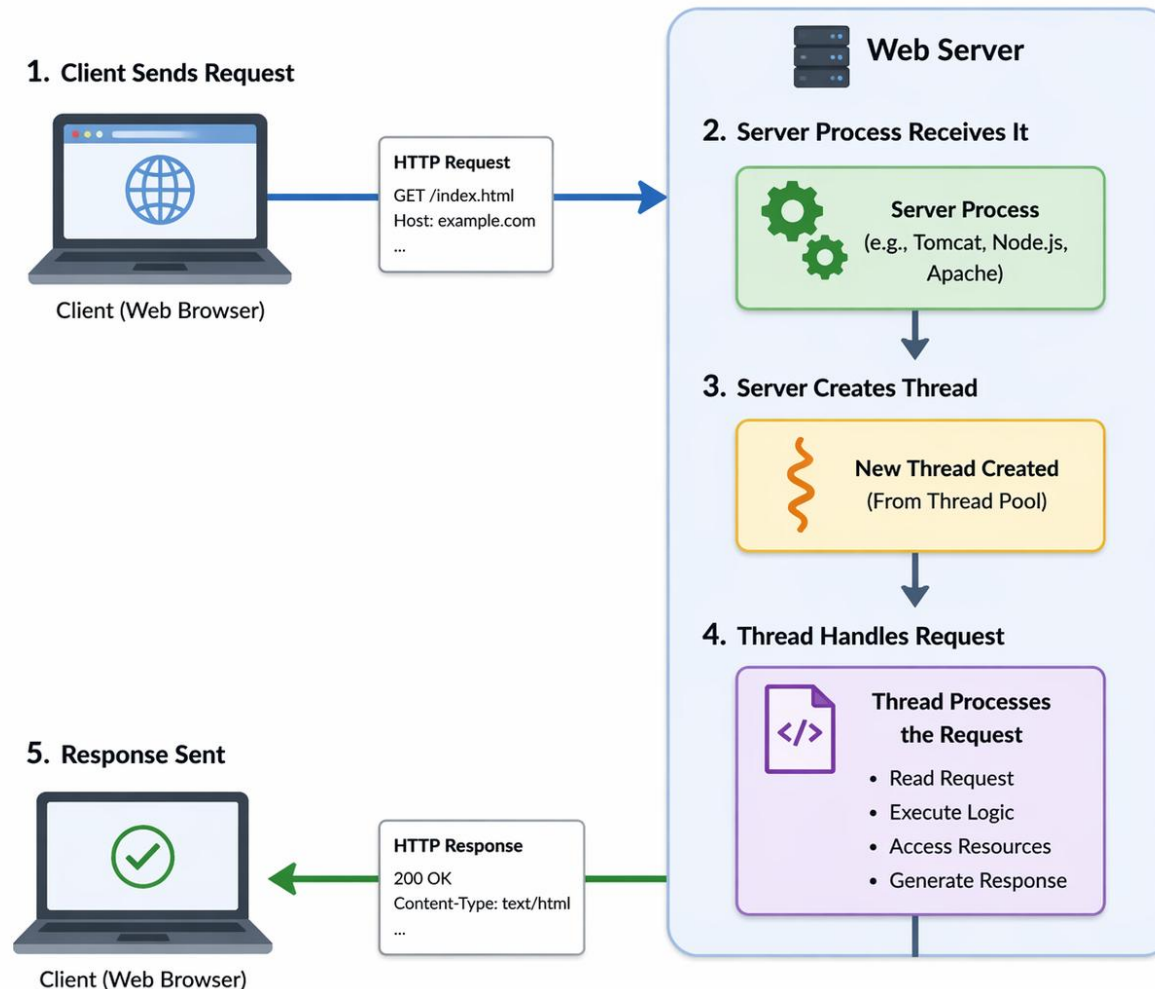
Basic Thread Workflow

Simple End-to-End

Web request

1. client send
2. server proc
3. server crea
4. thread han
5. response s

Each user request is
thread.



例

发送请求
进程接收
创建线程
处理请求

常由线程处理。

Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线程]

进程与线程

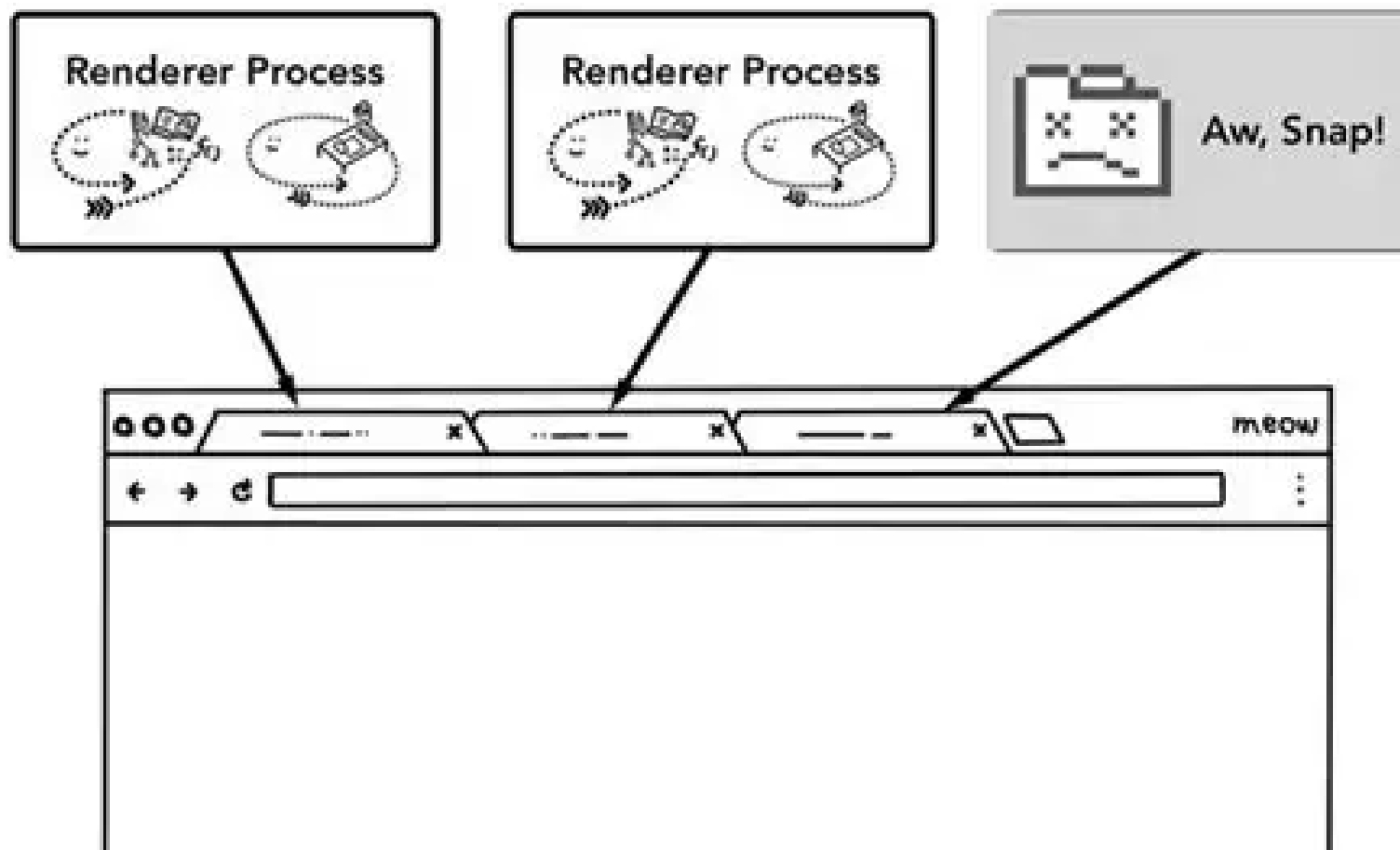
Process vs. Thread

Process	Thread
A process is the instance of a program executed by one or many threads.	A thread is a basic unit of CPU utilisation, which consists of its own thread ID, a program counter, a register and a stack.
A process may contain multiple threads depending on the operating system.	A thread is the smallest unit of execution within a process.
In a multiprocessing environment, multiple processes do not share resources such as memory with each other.	Multiple threads of a given process running concurrently can share resources such as memory with each other.
Processes take more time for context switching.	Threads take less time for context switching.
Processes take more time for creation and termination.	Threads take lesser time for creation and termination.
Processes consume more resources.	Threads consume fewer resources.
No other process can execute until the first process gets unblocked.	Another thread in the same task can run while one thread is blocked and waiting.
Process communication is complex.	Thread communication is much easier and efficient.

Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线程]

进程与线程

Process vs. Thread



快速回顾

Quick recap

- ❑ Process = a running program
- ❑ Thread = a unit of execution inside a process
- ❑ One process can have multiple threads
- ❑ Processes are independent and have separate memory
- ❑ Threads share memory and are lighter/faster than processes
- ❑ Why this matters in distributed systems
 - ❑ Servers often need to handle many requests at the same time
 - ❑ Threads help one server process multiple client requests concurrently
- ❑ 进程 = 一个正在运行的程序
- ❑ 线程 = 进程内的执行单元
- ❑ 一个进程可以有多个线程
- ❑ 进程是独立的，拥有独立的内存
- ❑ 线程共享内存，且比进程更轻且更快
- ❑ 为什么这在分布式系统中很重要
 - ❑ 服务器通常需要同时处理大量请求
 - ❑ 线程帮助一台服务器同时处理多个客户端请求

分布式系统中的多线程

Multithreading in Distributed Systems

Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

北京工业大学 分布式系统的原理
Beijing University of Technology, Principles of Distributed Systems

分布式系统中的多线程

Multithreading in Distributed Systems

Learning objectives

- ☐ explain what multithreading means
- ☐ understand why multithreading is important in distributed systems
- ☐ compare single-threaded and multi-threaded servers
- ☐ understand basic thread-related problems like:
 - ☐ race conditions
 - ☐ shared memory issues
 - ☐ need for synchronization

学习目标

- ☐ 解释一下多线程的含义
- ☐ 理解多线程在分布式系统中为何重要
- ☐ 比较单线程和多线程服务器
- ☐ 理解基本的线程相关问题，比如：
 - ☐ 比赛条件
 - ☐ 共享记忆问题
 - ☐ 同步需求

热身

Warm-up

If 100 users connect to one server at the same time, should the server handle them one by one or together?

Expected answer:

Together

or at least as many as possible at the same time

That is why multithreading is important.

如果100名用户同时连接一台服务器，服务器应该一个一个处理还是一起处理？

预期答案：

一起

或者至少同时拥有尽可能多的数量

这就是为什么多线程很重要。

什么是多线程？

What is multithreading?

Simple definition

Multithreading means:
one process uses multiple threads to
do multiple tasks at the same time.

Multithreading = many workers inside
one program.

简单定义

多线程意味着：
一个进程使用多个线程同时执行多个任务。

多线程 = 一个程序内有多个工作者。

什么是多线程?

What is multithreading?

Distributed systems often have:

- ☐ many users
- ☐ many requests
- ☐ many tasks happening together

A server may need to:

- ☐ accept connections
- ☐ read requests
- ☐ send replies
- ☐ write logs
- ☐ access database
- ☐ monitor timeouts

If one thread does everything, the system becomes slow.

分布式系统通常具备:

- ☐ 许多用户
- ☐ 许多请求
- ☐ 许多任务同时进行

服务器可能需要:

- ☐ 接受连接
- ☐ 读取请求
- ☐ 发送回复
- ☐ 写入日志
- ☐ 访问数据库
- ☐ 监控超时

如果一个线程做了所有事情, 系统就会变慢。

单线程服务器

Single-threaded server

Simple idea

A single-threaded server handles one client at a time.

简单的想法

单线程服务器一次处理一个客户端。

Flow

- ☐ client 1 connects
- ☐ server handles client 1
- ☐ client 2 must wait
- ☐ client 3 must wait
- ☐ only after client 1 finishes does the next client start

流

- ☐ 客户端1连接
- ☐ 服务器处理客户端1
- ☐ 客户端2必须等待
- ☐ 客户端3必须等待
- ☐ 只有客户端1完成后，下一个客户端才开始

Single-threaded = one worker serving one customer at a time.

单线程=一名工人一次服务一个客户。

单线程服务器的问题

Problems with a single-threaded server

1. Slow response

If one client takes a long time, all others must wait.

2. Poor scalability

More clients means longer waiting lines.

3. Wasted concurrency

Modern systems often have multiple CPU cores, but one thread may not use them effectively.

4. Bad user experience

Users may see delays, freezing, or timeouts.

1. 反应缓慢

如果一个客户需要很长时间，其他所有客户都必须等待。

2. 可扩展性差

客户越多，等待队伍越长。

3. 浪费并发

现代系统通常拥有多个CPU核心，但单个线程可能无法有效利用它们。

4. 糟糕的用户体验

用户可能会遇到延迟、冻结或超时的情况。

多线程服务器

Multi-threaded server

Simple idea

A multi-threaded server creates separate threads so that multiple clients can be handled at the same time.

Flow

1. client 1 connects → thread 1 handles it
2. client 2 connects → thread 2 handles it
3. client 3 connects → thread 3 handles it
4. main server thread keeps accepting new clients

Multi-threaded = many workers serving many customers.

简单的想法

多线程服务器创建独立线程，以便同时处理多个客户端。

流

1. 客户端1连接→线程1负责处理
2. 客户端2连接→线程2负责处理
3. 客户端3连接→线程3负责处理
4. 主服务器线程不断接受新客户端

多线程=多用户服务多客户。

为什么多线程能改善分布式系统

Why multithreading improves distributed systems

1. Better responsiveness

One slow client does not block all others.

2. Better concurrency

Many clients can be served together.

3. Better resource usage

CPU and I/O waiting can overlap.

4. More realistic for network servers

Most real servers serve many users at once.

1. 响应更快

一个慢客户端不会阻断所有其他客户端。

2. 更好的并发

许多客户可以一起被送达。

3. 更好的资源利用

CPU和I/O等待可能会重叠。

4. 更适合网络服务器

大多数真实服务器同时服务多个用户。

现实世界的例子

Real-world examples

Example 1: Web server

Many users open a website at the same time.
A thread can handle each request or a pool of threads can handle many requests.

Example 2: WeChat / messaging system

The app/server may need to:

send messages

receive messages

update notifications

handle media upload

These tasks often happen concurrently.

示例1：网页服务器

许多用户同时打开网站。

一个线程可以处理每个请求，或者一个线程池可以处理多个请求。

例子2：微信/消息系统

应用/服务器可能需要：

发送消息

接收消息

更新通知

处理媒体上传

这些任务通常同时进行。

现实世界的例子

Real-world examples

Example 3: Online shopping system

At the same time, a system may:

- search products
- check login
- update cart
- handle payments
- end notification

Multiple threads help manage these parallel tasks.

示例3：在线购物系统

同时，一个系统可能：

- 检索产品
- 检查登录
- 更新卡
- 处理付款
- 结束通知

多线程帮助管理这些并行任务。

基本服务器线程模型

Basic server threading model

Thread roles

A simple threaded server may have:

Main thread

- starts server
- listens for new connections
- creates worker threads

Worker threads

- handle client requests
- process data
- send responses

Optional background threads

- Logging
- Monitoring
- cleanup

线程角色

一个简单的线程服务器可能具备：

主线

- 启动服务器
- 倾听新的联系
- 创建工作线程

工作线程

- 处理客户端请求
- 工艺数据
- 发送回复

可选的背景线程

- 伐木
- 监测
- 清理工作

常见的多线程模型

Common multithreading models

1. One thread per client

Each client gets its own thread.

Advantage

Easy to understand.

Disadvantage

Too many clients may create too many threads.

2. Thread pool

A fixed number of worker threads are created in advance.

Advantage

Better control of resources.

Disadvantage

If all threads are busy, new clients wait.

1. 每个客户端一个线程

每个客户端都有自己的线程。

优势

很容易理解。

缺点

客户端过多可能会创建过多线程。

2. 线程池

事先创建固定数量的工作线程。

优势

更好地控制资源。

缺点

如果所有线程都忙，新客户端就会等待。

重要的线程问题

Important thread problems

A. Shared memory

Threads in the same process often share memory.

Good side

Easy to share data.

Bad side

Can cause conflicts.

共享内存

同一进程中的线程通常共享内存。

好的一面

数据共享简便。

坏的一面

可能会引发冲突。

B. Race condition

A race condition happens when:

two or more threads access the same data and at least one thread changes it without proper coordination

B. 竞赛条件

当以下情况发生竞赛条件:

两个或多个线程访问同一数据, 且至少有一个线程在未适当协调的情况下更改了数据

Simple example

Two threads update the same counter at the same time.

Expected:

counter becomes 2

Possible wrong result:

counter becomes 1

简单示例

两个线程同时更新同一个计数器。

预期:

计数器变为2

可能的错误结果:

计数器变为1

重要的螺纹问题

Important thread problems

C. Synchronization

Synchronization means using rules/tools so threads do not interfere incorrectly.

Examples:

Locks mutexes semaphores
synchronized sections

Synchronization = traffic rules for threads.

C. 同步

同步意味着使用规则/工具，避免线程错误干扰。

示例：

锁、互斥、信号量、同步段

同步 = 线程的流量规则。

比赛条件的简单类比

Easy analogy for race condition

Imagine two students editing the same WPS/Google Doc line at the same time without coordination.

One change may overwrite the other.
That is like a race condition.

想象一下，两个学生同时编辑同一行WPS/Google文档，没有协调。

一个变更可能会覆盖另一个。
这就像是比赛条件。

比赛条件的简单类比

Easy analogy for race condition

Suppose balance = 100
Thread A withdraws 30
Thread B withdraws 50

假设平衡=100
线程A撤回30
线程B撤回50

If both read the old value at the same time, the final result may be wrong.

如果两者同时读取旧值，最终结果可能不准确。

This shows why thread safety matters.

这说明了线程安全的重要性。

螺纹安全

Thread safety

Simple definition

A piece of code is thread-safe if it works correctly even when multiple threads run it at the same time.

Thread-safe = safe even when many threads use it together.

简单定义

如果一段代码在多个线程同时运行时也能正常工作，则该代码是线程安全的。

线程安全=即使多个线程同时使用也安全。

分布式系统中的多线程

Multithreading in distributed systems specifically

Distributed systems already have complexity because:

- different machines communicate
- network delays exist
- failures happen

Multithreading adds another layer:

- ❑ multiple tasks run inside one process
- ❑ timing becomes harder to predict bugs can become subtle
- ❑ So multithreading improves performance, but also increases complexity.

分布式系统本身就具有复杂性，原因包括：

- 不同的机器会进行通信
- 网络存在延迟
- 失败是难免的

多线程增加了另一层：

- ❑ 多个任务在一个进程中运行
- ❑ 时机预测变得更难，漏洞也可能变得微妙
- ❑ 所以多线程提升了性能，但也增加了复杂度。

利弊与挑战

Benefits vs challenges

Benefits	Challenges
faster response	race conditions
better concurrency	shared memory bugs
handles many clients	synchronization needed
better system utilization	harder debugging

快速回顾

Quick recap

- ❑ Multithreading in Distributed Systems
- ❑ Multithreading = multiple threads inside one process
- ❑ Helps servers handle many clients at once
- ❑ Single-threaded servers are simple but slow
- ❑ Multi-threaded servers improve responsiveness
- ❑ Threads share memory, so problems can happen:
 - ❑ race conditions
 - ❑ shared data conflicts
 - ❑ synchronization needed

Takeaway

- ❑ Multithreading improves performance, but correctness becomes more difficult.

- ❑ 分布式系统中的多线程
- ❑ 多线程 = 一个进程中的多个线程
- ❑ 帮助服务器同时处理多个客户端
- ❑ 单线程服务器简单但速度较慢
- ❑ 多线程服务器提升响应速度
- ❑ 线程共享内存，因此可能出现以下问题：
 - ❑ 比赛条件
 - ❑ 共享数据冲突
 - ❑ 需要同步

摘要

- ❑ 多线程提升了性能，但正确性变得更难。

分布式系统中的虚拟化

Virtualization in Distributed Systems

Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线程]

北京工业大学 分布式系统的原理
Beijing University of Technology, Principles of Distributed Systems

分布式系统中的虚拟化

Virtualization in Distributed Systems

Can one physical computer behave like many computers?

一台物理计算机能像许多计算机一样表现吗？

Yes — that is one of the main ideas of virtualization.

是的——这正是虚拟化的主要理念之一。

什么是虚拟化？

What is virtualization?

Simple definition

Virtualization means using software to create a virtual version of computing resources.

This may include:

- ☐ virtual computers
- ☐ virtual storage
- ☐ virtual networks
- ☐ virtual servers

Virtualization = one real computer can act like many computers.

简单定义

虚拟化是指使用软件创建计算资源的虚拟版本。

这可能包括：

- ☐ 虚拟计算机
- ☐ 虚拟存储
- ☐ 虚拟网络
- ☐ 虚拟服务器

虚拟化=一台真实计算机可以像多台计算机一样工作。

为什么虚拟化很重要

Why virtualization matters

Without virtualization:

one physical machine usually runs one main operating environment

With virtualization:

one physical machine can run multiple isolated environments

This is useful because:

- ☐ resources are used more efficiently
- ☐ systems are easier to test
- ☐ services are easier to deploy
- ☐ distributed systems can be simulated cheaply

不使用虚拟化:

一台物理机器通常运行一个主要的操作环境

使用虚拟化:

一台物理机器可以运行多个隔离环境

这很有用，因为:

- ☐ 资源的利用更高效
- ☐ 系统更容易测试
- ☐ 服务更容易部署
- ☐ 分布式系统可以廉价地模拟

物理机与虚拟机

Physical machine vs virtual machine

Physical machine

A real computer with:

- ☐ CPU
- ☐ Memory
- ☐ Disk
- ☐ network card

Virtual machine

A software-based computer running on top of a physical machine.

A VM can have:

- ☐ its own OS
- ☐ its own IP address
- ☐ its own applications
- ☐ its own files

A VM looks like a real computer, even though it is software.

物理机器

一台真正的电脑，具备：

- ☐ 中央处理器
- ☐ 记忆
- ☐ 圆盘
- ☐ 网络卡

虚拟机

一台运行在物理机器上的软件计算机。

虚拟机可以包含：

- ☐ 它自己的操作系统
- ☐ 它自己的IP地址
- ☐ 其自身应用
- ☐ 它自己的文件

虚拟机看起来像一台真实的电脑，尽管它是软件。

虚拟机监控程序

Hypervisor

A hypervisor is the software that allows virtual machines to run.

It manages:

- ☐ CPU allocation
- ☐ memory allocation
- ☐ storage access
- ☐ Networking

Hypervisor = manager of virtual machines.

Easy analogy

Think of a hotel building.

physical machine = hotel building

virtual machines = rooms

hypervisor = hotel manager deciding who gets which room and resources

虚拟机监控程序是允许虚拟机运行的软件。

其管理功能包括：

- ☐ CPU 分配
- ☐ 内存分配
- ☐ 存储访问
- ☐ 人脉网络

虚拟机管理程序 = 虚拟机管理器。

简单的比喻

想象一栋酒店建筑。

物理机器 = 酒店大楼

虚拟机 = 房间

虚拟机监控器 = 酒店经理决定谁拥有哪个房间和资源

基本虚拟机架构

Basic VM architecture

Hardware → Hypervisor → Multiple VMs

Each VM contains:

- ☐ guest OS
- ☐ Libraries
- ☐ applications

多虚拟机→硬件→虚拟机监控程序

每个虚拟机包含:

- ☐ 客体操作系统
- ☐ 图书馆
- ☐ 应用

分布式系统为何使用虚拟化

Why distributed systems use virtualization

Distributed systems often need:

- ☐ multiple servers
- ☐ isolated services
- ☐ scalable deployment
- ☐ easy testing

Virtualization helps by allowing:

- ☐ many virtual nodes on fewer physical machines
- ☐ quick creation of new environments
- ☐ safe testing of distributed programs

分布式系统通常需要:

- ☐ 多服务器
- ☐ 孤立服务
- ☐ 可扩展部署
- ☐ 简易测试

虚拟化通过以下方式帮助:

- ☐ 在较少的物理机器上拥有许多虚拟节点
- ☐ 快速创建新环境
- ☐ 分布式程序的安全测试

虚拟化的主要优势

Main benefits of virtualization

1. Better resource utilization

A single physical server can host multiple virtual systems.

Example

Instead of 10 underused physical servers, one stronger machine can run several VMs.

2. Isolation

Each VM is separated from the others.

Why useful?

If one VM crashes, others may continue working.

Isolation = one problem does not automatically destroy everything else.

1. 更好的资源利用

单个物理服务器可以承载多个虚拟系统。

示例

一台性能更强的机器可以运行多个虚拟机，而不是10台闲置的物理服务器。

2. 隔离

每个虚拟机都与其他虚拟机分开。

为什么有用？

如果一个虚拟机崩溃，其他虚拟机可能会继续工作。

隔离=一个问题不会自动毁掉其他所有问题。

虚拟化的主要优势

Main benefits of virtualization

3. Easy deployment

You can quickly create, copy, move, or delete VMs.

Example

A new test server can be created in minutes.

3. 易于部署

你可以快速创建、复制、移动或删除虚拟机。

示例

新的测试服务器可以在几分钟内建立。

4. Easy testing

Students can simulate multiple machines on one laptop or lab machine.

Example

Run client on one VM and server on another VM.

4. 简便测试

学生可以在一台笔记本电脑或实验机上模拟多台机器。

示例

在一个虚拟机上运行客户端，在另一个虚拟机上运行服务器。

虚拟化的主要优势

Main benefits of virtualization

5. Scalability support

Cloud systems can create more virtual machines when demand grows.

5. 可扩展性支持

当需求增长时，云系统可以创建更多虚拟机。

6. Portability

A VM image can be moved from one host machine to another.

6. 便携性

虚拟机镜像可以从一台主机转移到另一台主机。

虚拟化与分布式系统

Virtualization vs distributed systems

Important clarification:

Virtualization is not the same as distributed systems.

Difference:

Virtualization = how computing resources are created and managed

Distributed systems = how multiple computers or nodes cooperate over a network

But:

Virtualization helps build and run distributed systems more easily.

重要澄清：

虚拟化不同于分布式系统。

区别：

虚拟化=计算资源的创建和管理方式

分布式系统 = 多台计算机或节点如何在网络上协作

但是：

虚拟化有助于更轻松地构建和运行分布式系统。

什么是容器？

What is a container?

简单定义

容器是一种轻量级的隔离环境，用于运行应用程序。
与虚拟机不同，容器通常不包含完整的访客操作系统。

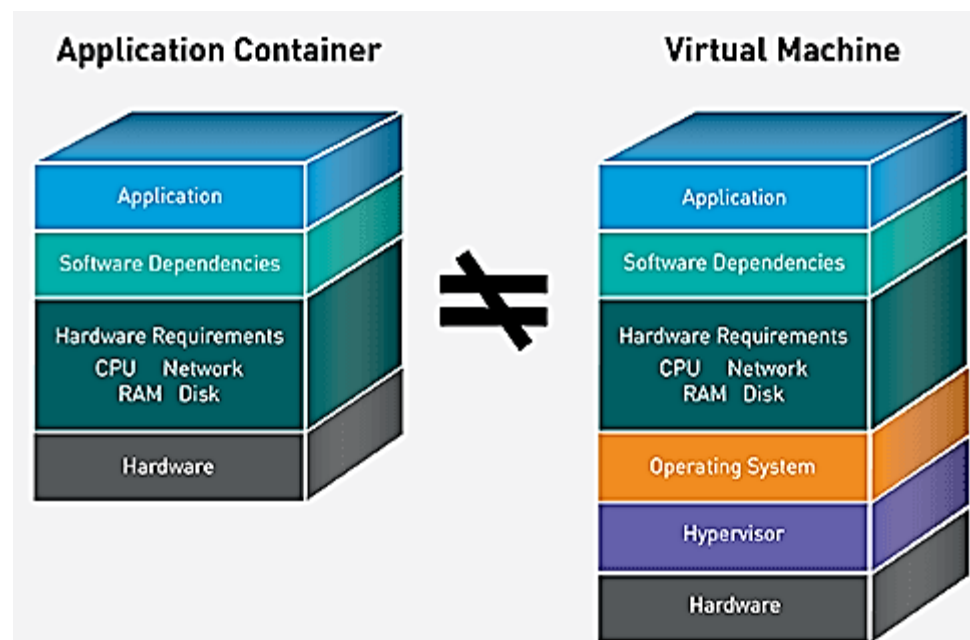
容器=比虚拟机轻。

Simple definition

A container is a lightweight isolated environment for running applications.

Unlike a VM, a container does not usually include a full guest operating system.

Container = lighter than a VM.



虚拟机与容器简单比较

VM vs Container Simple comparison

Feature	Virtual Machine	Container
OS	full guest OS	shares host OS kernel
size	larger	smaller
startup	slower	faster
isolation	strong	lighter
use	full system simulation	lightweight app deployment

Easy analogy

VM = full apartment

container = a room inside a building

Or:

VM = full house with all furniture

container = one prepared workspace inside a big building

简单的比喻

VM = 完整公寓

容器 = 建筑物内的房间

或者:

VM = 全屋, 全部家具

容器 = 大型建筑内一个已准备好的工作区

什么时候用虚拟机，什么时候用容器

When to use VMs vs containers

Use VMs when:

- ☐ you need full OS isolation
- ☐ you want to simulate separate machines
- ☐ you need different operating systems

Use containers when:

- ☐ you want lightweight deployment
- ☐ you want fast startup
- ☐ you need to package an application simply

在以下情况下使用虚拟机:

- ☐ 你需要完全的操作系统隔离
- ☐ 你想模拟不同的机器
- ☐ 你需要不同的操作系统

以下情况下使用容器:

- ☐ 你需要轻量化部署
- ☐ 你想要快速启动
- ☐ 你需要简单地打包一个应用程序

为什么虚拟机网络在分布式系统中很重要

Why VM networking matters in distributed systems

Distributed systems depend on:

- ❑ machines communicating over a network
- ❑ separate nodes sending requests and responses
- ❑ VM networking lets simulate this without needing many physical computers.

VM lets us practice distributed systems on one real machine.

分布式系统依赖于:

- ❑ 通过网络通信的机器
- ❑ 发送请求和响应的独立节点
- ❑ 虚拟机网络让无需大量实体计算机即可模拟这些。

虚拟机网络让我们可以在一台真实机器上练习分布式系统。

简单虚拟化的实际场景

Simple virtualization practical scenario

Example setup

VM1 = Server

VM2 = Client

- ☐ create two VMs
- ☐ connect them on same virtual network
- ☐ find IP addresses
- ☐ run server on VM1 run client on VM2 send request and receive response
- ☐ This simulates a real distributed environment.

示例设置

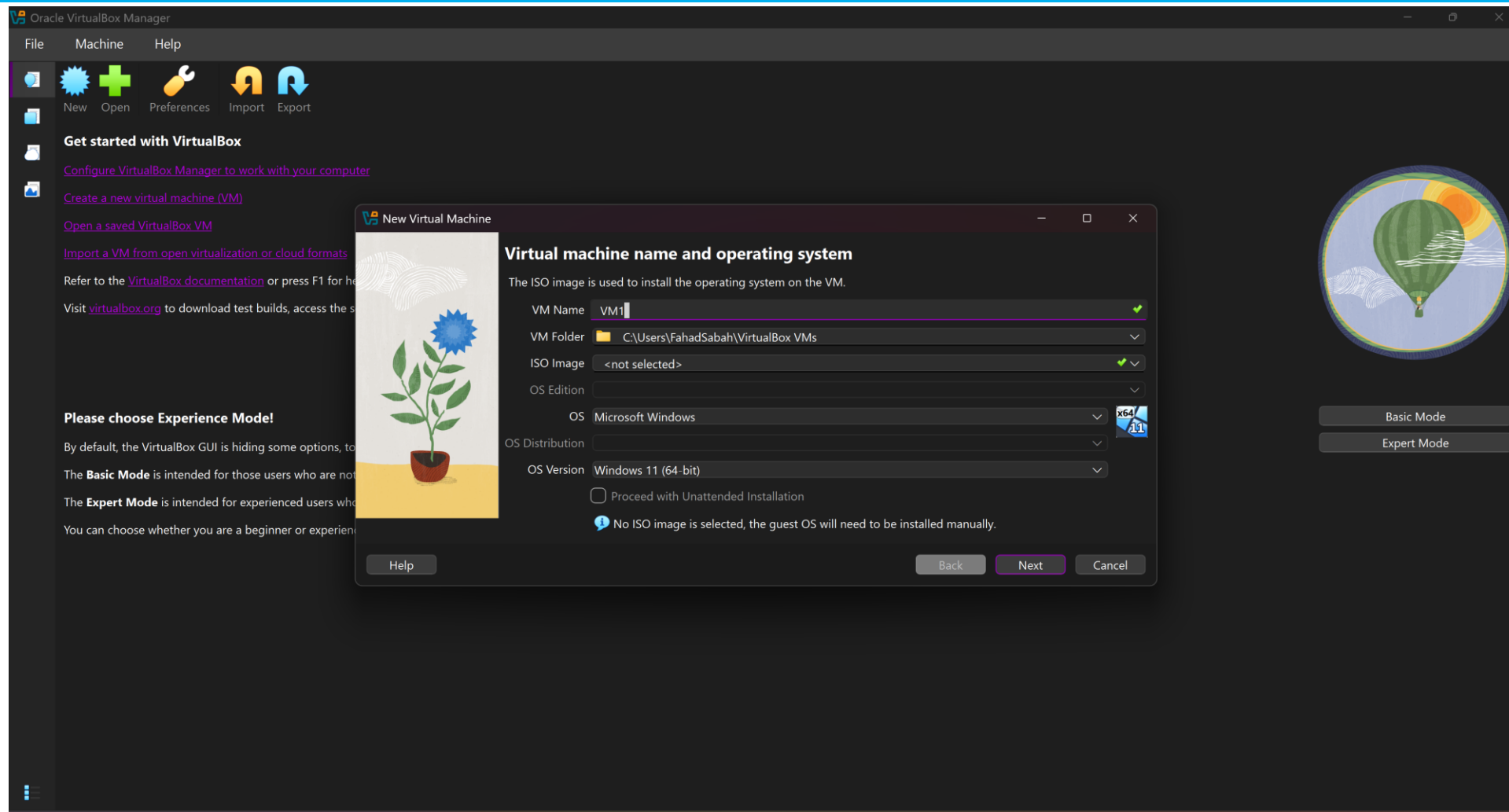
VM1 = Server

VM2 = Client

- ☐ 创建两个虚拟机
- ☐ 它们连接到同一个虚拟网络
- ☐ 查找IP地址
- ☐ 在 VM1 上运行服务器，运行客户端，在 VM2 上运行请求并接收响应
- ☐ 这模拟了一个真实的分布式环境。

简单虚拟化的实际场景

Simple virtualization practical scenario



Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线程]

虚拟化的优势

Virtualization benefits

Virtualization allows to:

- ☐ simulate many machines
- ☐ test distributed systems cheaply
- ☐ experiment safely
- ☐ reset systems easily
- ☐ practice networking concepts

虚拟化允许:

- ☐ 模拟许多机器
- ☐ 廉价测试分布式系统
- ☐ 安全实验
- ☐ 系统重置轻松
- ☐ 练习网络概念

虚拟化的局限性/挑战

Limitations / challenges of virtualization

1. Overhead

VMs still use CPU, memory, and storage.

2. Performance may not equal

Virtualized performance can be slightly lower.

3. Complexity

Managing multiple VMs, IPs, and network settings can be confusing for beginners.

4. Resource limits

A laptop may not handle too many VMs at once.

1. 开销

虚拟机仍然需要使用CPU、内存和存储。

2. 性能可能不等

虚拟化性能可能略低。

3. 复杂性

管理多个虚拟机、IP 和网络设置对初学者来说可能会很困惑。

4. 资源限制

笔记本可能无法同时处理太多虚拟机。

快速回顾

Quick recap

Virtualization in Distributed Systems

- ☐ Virtualization lets one physical machine run many virtual systems
- ☐ VMs behave like full computers
- ☐ Containers are lighter-weight isolated environments
- ☐ Virtualization helps with:
 - ☐ Testing
 - ☐ Deployment
 - ☐ Isolation
 - ☐ Scaling
 - ☐ VM networking allows distributed systems to be simulated

Takeaway

Virtualization helps us create and test distributed systems more easily and efficiently.

分布式系统中的虚拟化

- ☐ 虚拟化让一台物理机器能够运行多个虚拟系统
- ☐ 虚拟机的表现就像完整的计算机
- ☐ 集装箱是较轻的隔离环境
- ☐ 虚拟化有助于:
 - ☐ 测试
 - ☐ 部署
 - ☐ 隔离
 - ☐ 缩放
 - ☐ 虚拟机网络允许对分布式系统进行模拟

摘要

虚拟化帮助我们更轻松高效地创建和测试分布式系统。

期中项目讨论

Midterm Project Discussion

Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

北京工业大学 分布式系统的原理
Beijing University of Technology, Principles of Distributed Systems

项目讨论

Project Discussion

(TITLE-ABS-KEY ("Distributed System") AND TITLE-ABS-KEY ("Virtualization"))

Scopus

Search Sources SciVal ? Create account Sign in

Advanced query

Search within Article title, Abstract, Keywords

Search documents * "Distributed System"

AND

Search within Article title, Abstract, Keywords

Search documents "Virtualization"

Save search Set search alert Add search field Reset Search

Documents Preprints Secondary documents

536 documents found Analyze results

Refine search

Search within results

Filters

Year

Document title Authors Source Year Citations

Article • Open access

1 A fault-tolerant and load-balancing scheduler for independent tasks on cloud-based virtual machines Hagras, T., El-Sayed, G.A. Cluster Computing, 29(1), 61 2026 0

Show abstract Find-It! View at Publisher Related documents

Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

北京工业大学 分布式系统的原理
Beijing University of Technology, Principles of Distributed Systems

项目讨论

Project Discussion

(TITLE-ABS-KEY ("Distributed System") AND TITLE-ABS-KEY ("Virtualization")) AND PUBYEAR > 2006
AND PUBYEAR < 2027 AND (LIMIT-TO (LANGUAGE , "English"))

Scopus

Search Sources SciVal ? ? Create account Sign in

Advanced query ☐

Search within Article title, Abstract, Keywords

Search documents * "Distributed System"

AND

Search within Article title, Abstract, Keywords

Search documents "Virtualization"

Save search Set search alert + Add search field Reset Search

Documents Preprints Secondary documents

508 documents found Analyze results

Refine search Search within results

Filters Clear all Year Clear

1 All Export Download Citation overview ... More Show all abstracts Sort by Date (newest)

Document title	Authors	Source	Year	Citations
Article • Open access				
1 A fault-tolerant and load-balancing scheduler for independent tasks on cloud-based virtual machines	Hagrass, T., El-Sayed, G.A.	Cluster Computing, 29(1), 61	2026	0

Show abstract Find-It View at Publisher Related documents

Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

北京工业大学 分布式系统的原理
Beijing University of Technology, Principles of Distributed Systems

项目讨论

Project Discussion

(TITLE-ABS-KEY ("Distributed System") AND TITLE-ABS-KEY ("Virtualization")) AND PUBYEAR > 2006 AND PUBYEAR < 2027 AND (LIMIT-TO (LANGUAGE , "English")) AND (LIMIT-TO (DOCTYPE , "cp") OR LIMIT-TO (DOCTYPE , "ar"))

Document type ^

- ☐ Conference paper 307
- ☒ Article 140
- ☐ Conference review 31
- ☐ Book chapter 13
- ☐ Book 11

[Show all](#)

Source title v

Publication stage v

Keyword ^

- ☐ Distributed Systems 291
- ☐ Virtualization 194
- ☐ Cloud Computing 142

[Cancel](#) [Exclude](#) [Limit to](#)

 Scopus

Search Sources SciVal ?   FS

Advanced query ☐

(TITLE-ABS-KEY ("Distributed System") AND TITLE-ABS-KEY ("Virtualization")) AND PUBYEAR > 2006 AND PUBYEAR < 2027 AND (LIMIT-TO (LANGUAGE , "English")) AND (LIMIT-TO (DOCTYPE , "cp") OR LIMIT-TO (DOCTYPE , "ar"))

[Show less](#) 

[Save search](#) [Set search alert](#) [Edit in advanced search](#)

Beta

Documents Preprints Secondary documents

447 documents found [Analyze results](#)

Refine search ☐ All [Export](#) [Download](#) [Citation overview](#) [More](#) [Show all abstracts](#) Sort by [Date \(newest\)](#)  

Search within results

Filters [Clear all](#)

Year ^

☒ Range ☐ Individual

	Document title	Authors	Source	Year	Citations
☐ 1	Article • Open access A fault-tolerant and load-balancing scheduler for independent tasks on cloud-based virtual machines	Hagras, T., El-Sayed, G.A.	Cluster Computing , 29(1), 61	2026	0

[Show abstract](#) [Find it!](#) [View at Publisher](#) [Related documents](#)

Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

项目讨论

Project Discussion

Completeness of metadata -- 447 docs from Scopus

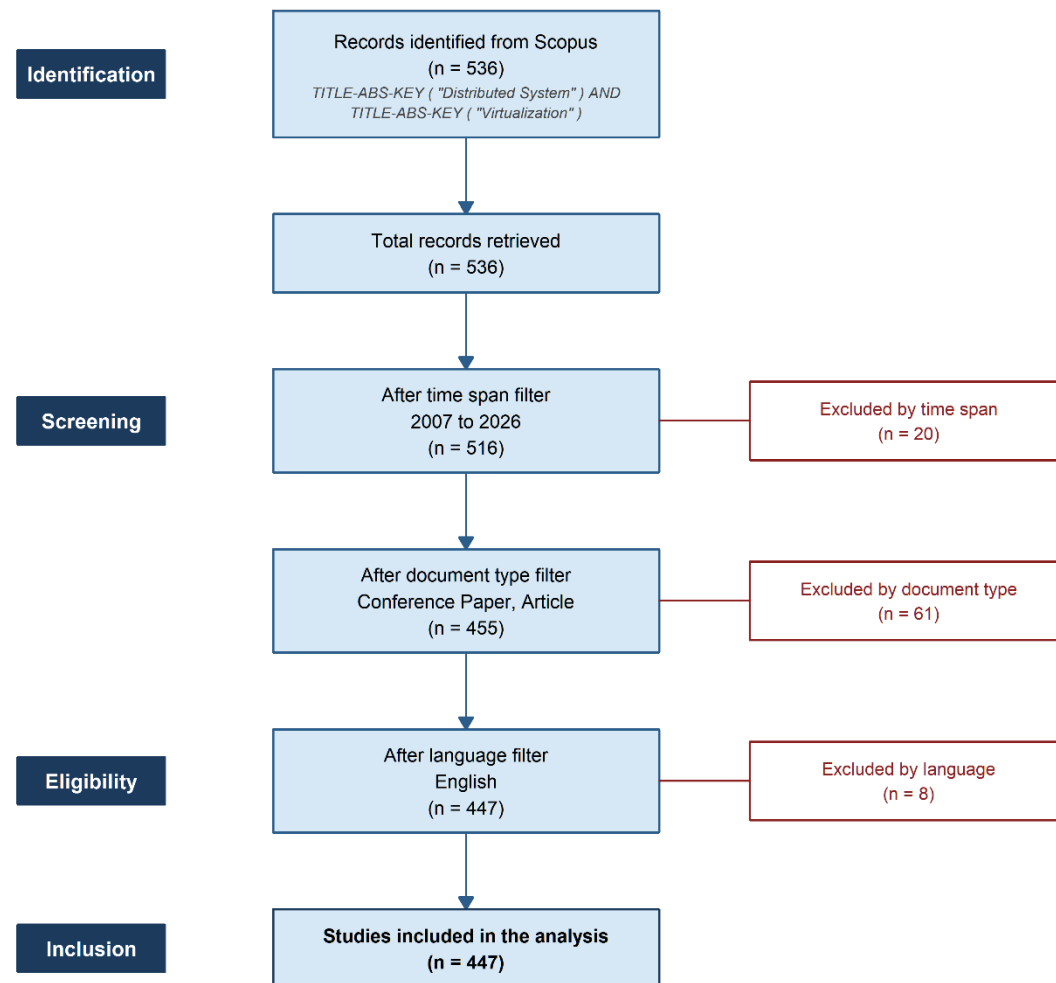
Metadata	Description	Missing Counts	Missing %	Trend	Status
AB	Abstract	0	0		Excellent
AU	Author	0	0		Excellent
DT	Document Type	0	0		Excellent
SO	Journal	0	0		Excellent
LA	Language	0	0		Excellent
PY	Publication Year	0	0		Excellent
TI	Title	0	0		Excellent
TC	Total Citation	0	0		Excellent

Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

北京工业大学 分布式系统的原理
Beijing University of Technology, Principles of Distributed Systems

棱镜流图

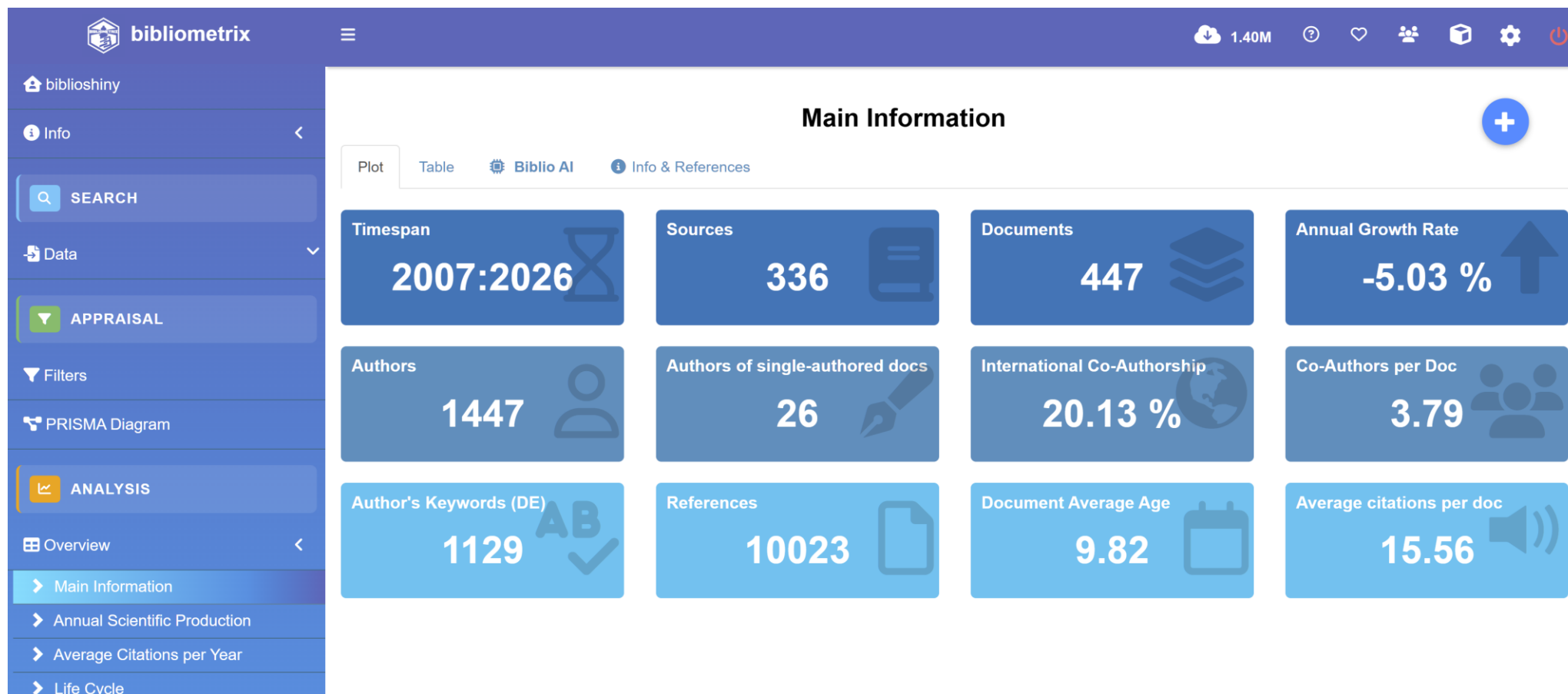
PRISMA Flow Diagram



Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

主要信息

Main Information

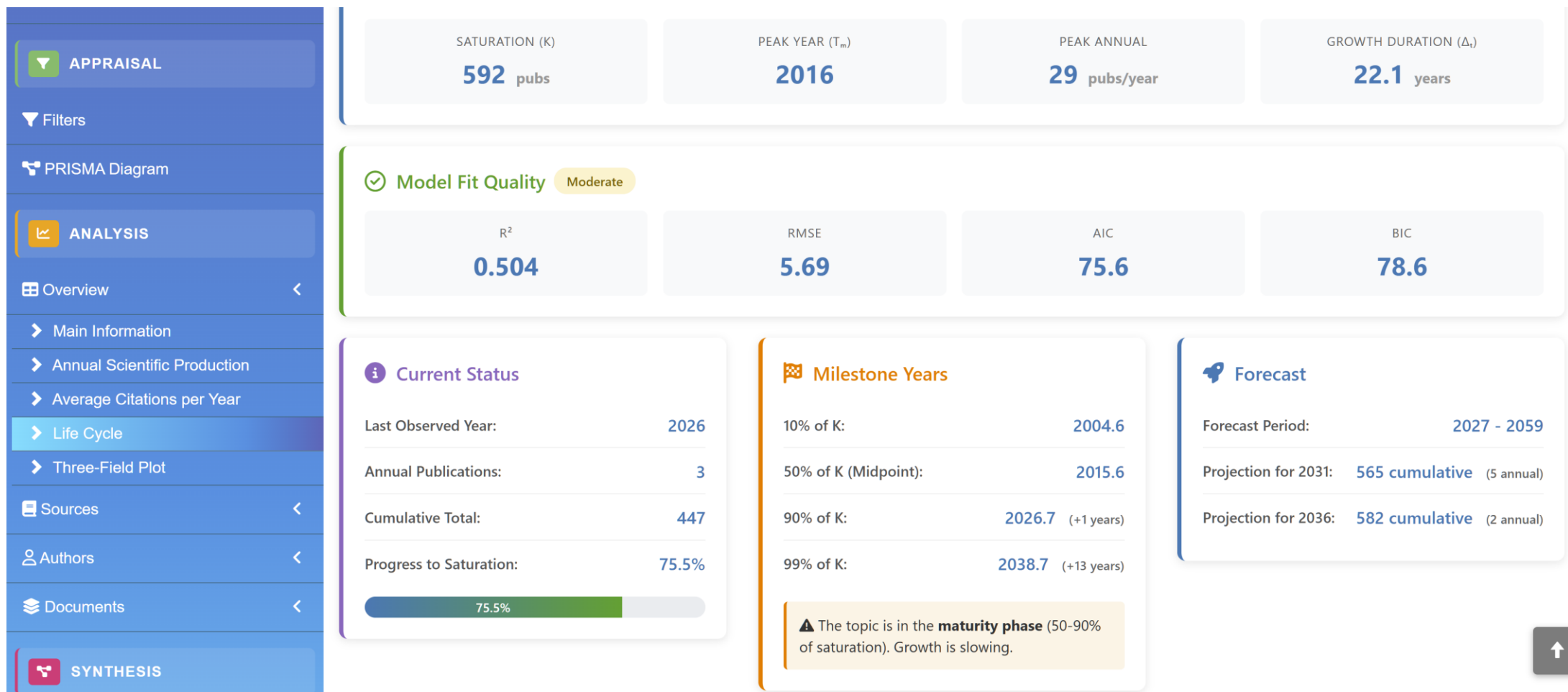


Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

北京工业大学 分布式系统的原理
Beijing University of Technology, Principles of Distributed Systems

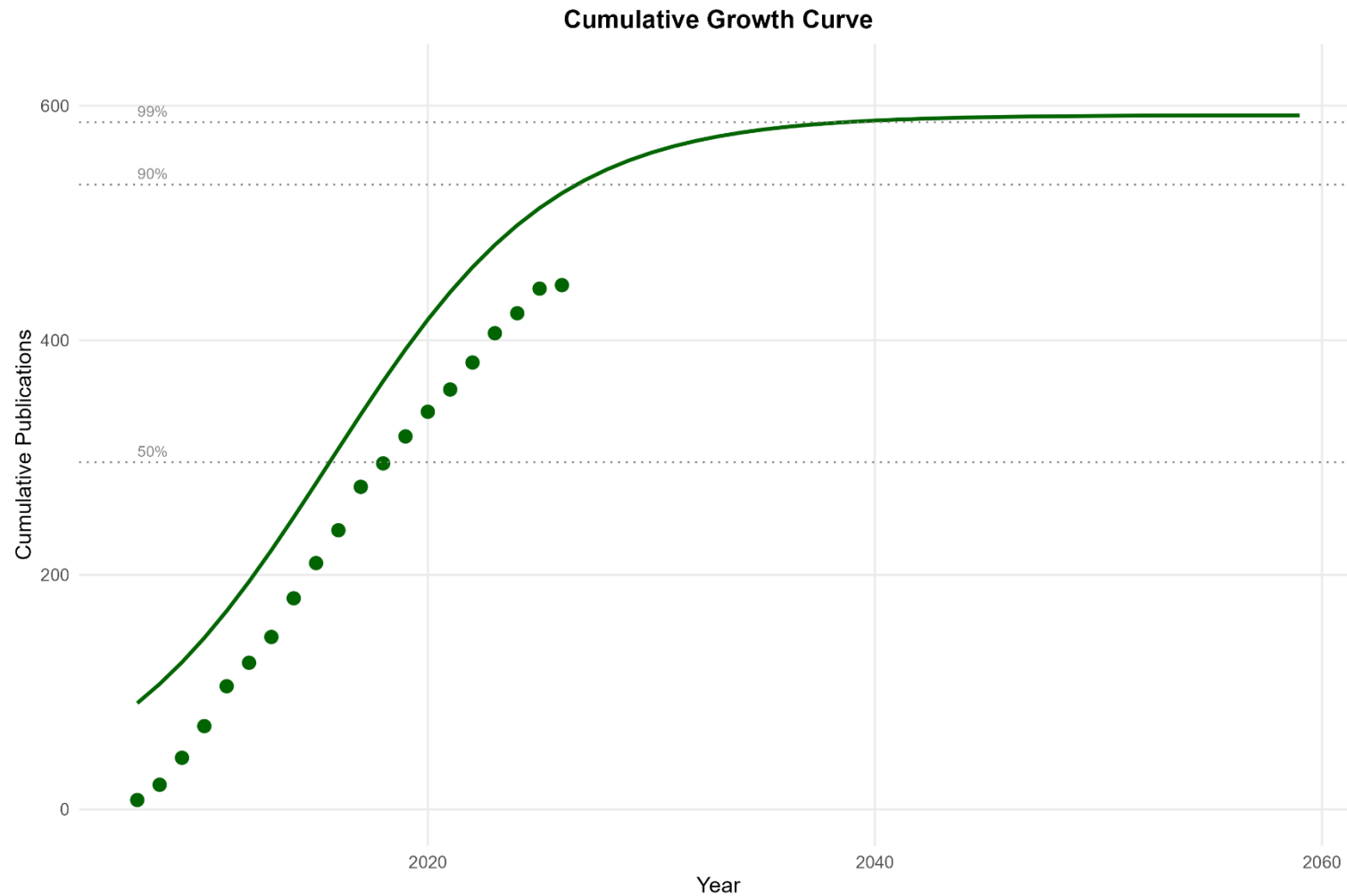
出版生命周期

Publication Life Cycle



Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线程]

出版生命周期 Publication Life Cycle



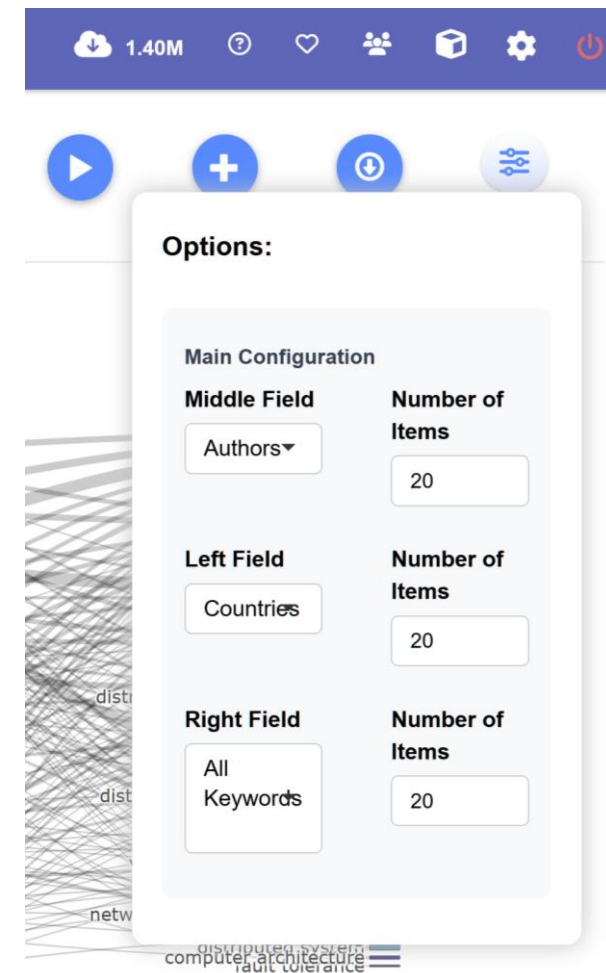
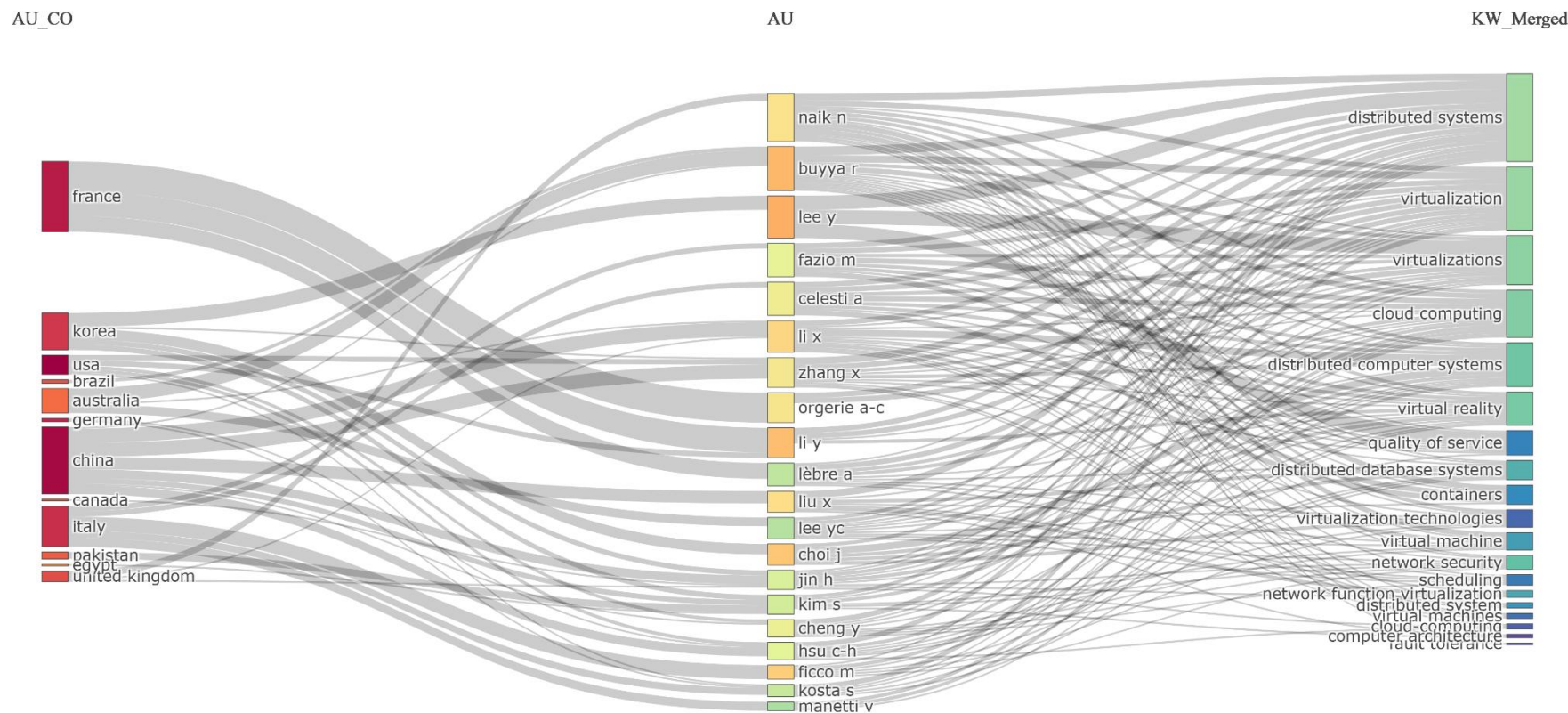
Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

北京工业大学 分布式系统的原理
Beijing University of Technology, Principles of Distributed Systems

三田地情

Three Field Plot

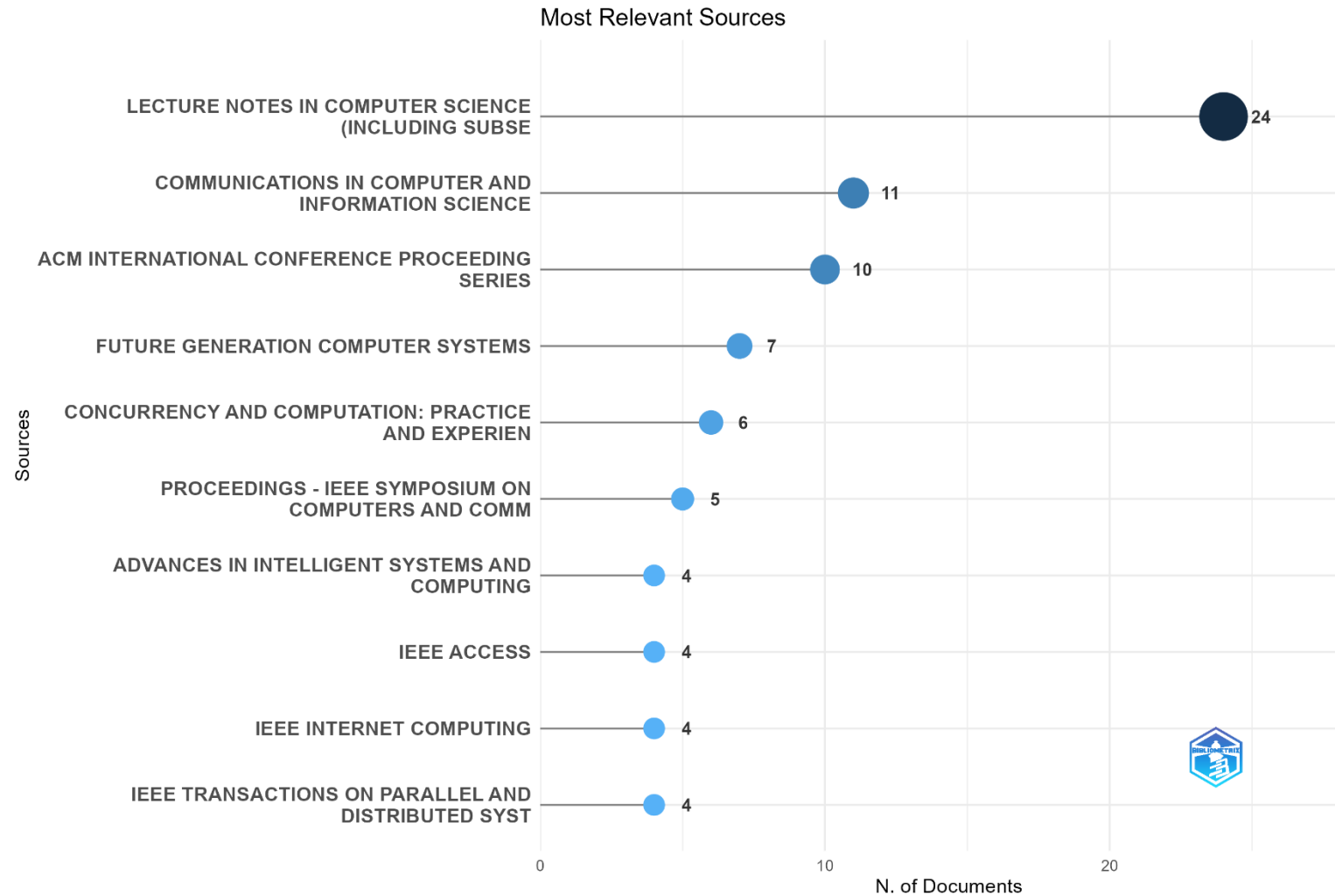
Authors' Countries → Authors → Keywords: Identifies national research specializations and thematic focuses



Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

最相关的资料来源

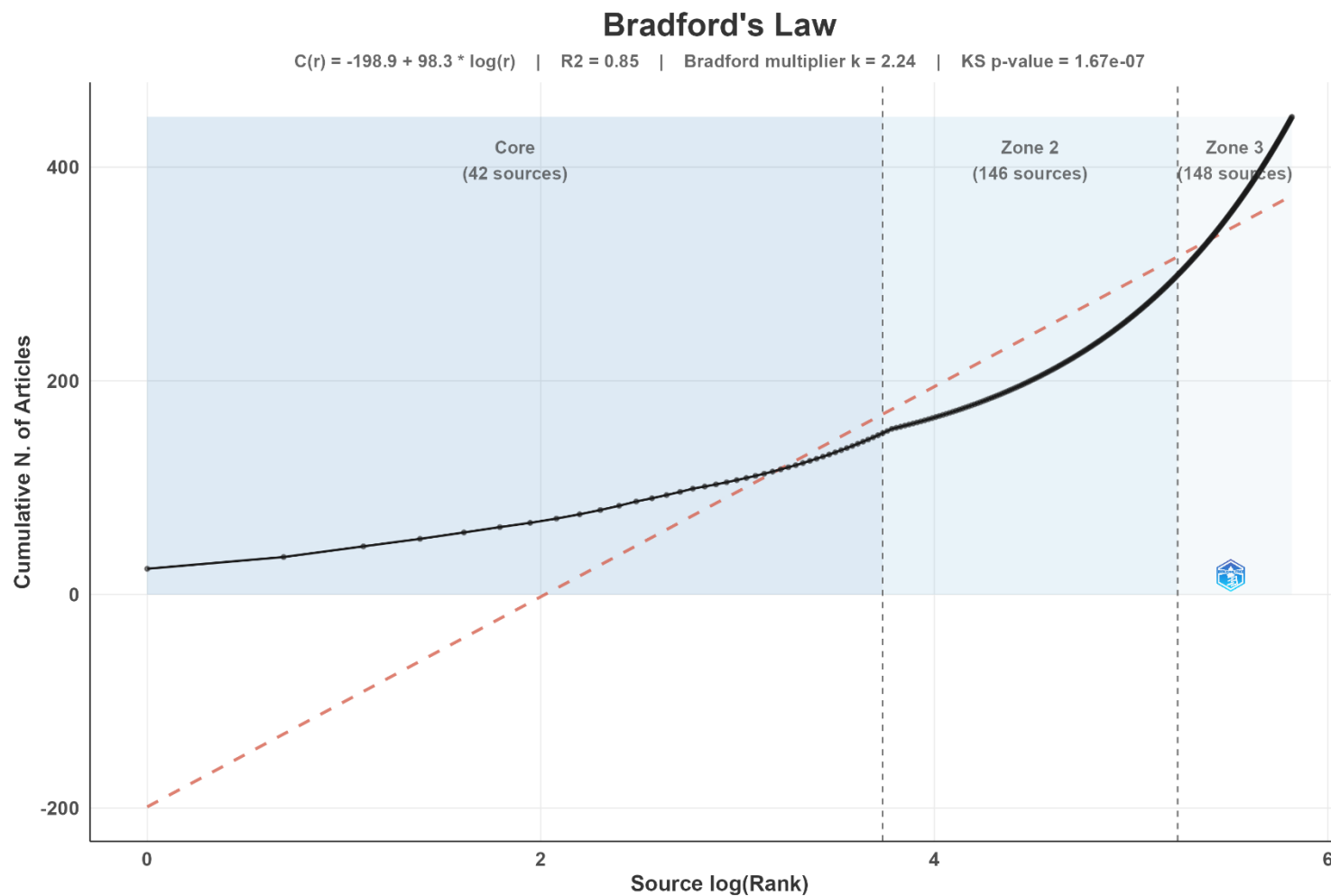
Most Relevant Sources



Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

布拉德福德散射定律

Bradford's Law of Scattering



Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

项目讨论

Project Discussion

The screenshot shows the Bibliometrix web application interface. The left sidebar contains navigation options: biblioshiny, Info, SEARCH, Data, APPRAISAL, Filters, PRISMA Diagram, ANALYSIS, Overview, Sources, Most Relevant Sources, Most Local Cited Sources, Bradford's Law (selected), Sources' Local Impact, and Sources' Production over Time. The main content area is titled 'Core Sources by Bradford's Law' and includes tabs for Plot, Table (selected), Summary, Biblio AI, and Info & References. A table displays the top 10 core sources, with columns for SO, Rank, Freq, cumFreq, and Zone. The table is filtered to show sources in Zone 1. The bottom of the table indicates 'Showing 1-10 of 336' sources.

SO	Rank	Freq	cumFreq	Zone
LECTURE NOTES IN COMPUTER SCIENCE (INCLUDING SUBSERIES LECTURE NOTES IN ARTIFICIAL INTELLIGENCE AND LECTURE NOTES IN BIOINFORMATICS)	1	24	24	Zone 1
COMMUNICATIONS IN COMPUTER AND INFORMATION SCIENCE	2	11	35	Zone 1
ACM INTERNATIONAL CONFERENCE PROCEEDING SERIES	3	10	45	Zone 1
FUTURE GENERATION COMPUTER SYSTEMS	4	7	52	Zone 1
CONCURRENCY AND COMPUTATION: PRACTICE AND EXPERIENCE	5	6	58	Zone 1
PROCEEDINGS - IEEE SYMPOSIUM ON COMPUTERS AND COMMUNICATIONS	6	5	63	Zone 1
ADVANCES IN INTELLIGENT SYSTEMS AND COMPUTING	7	4	67	Zone 1
IEEE ACCESS	8	4	71	Zone 1
IEEE INTERNET COMPUTING	9	4	75	Zone 1
IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS	10	4	79	Zone 1

Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

国家科学生产

Country's Scientific Production

Countries' Scientific Production

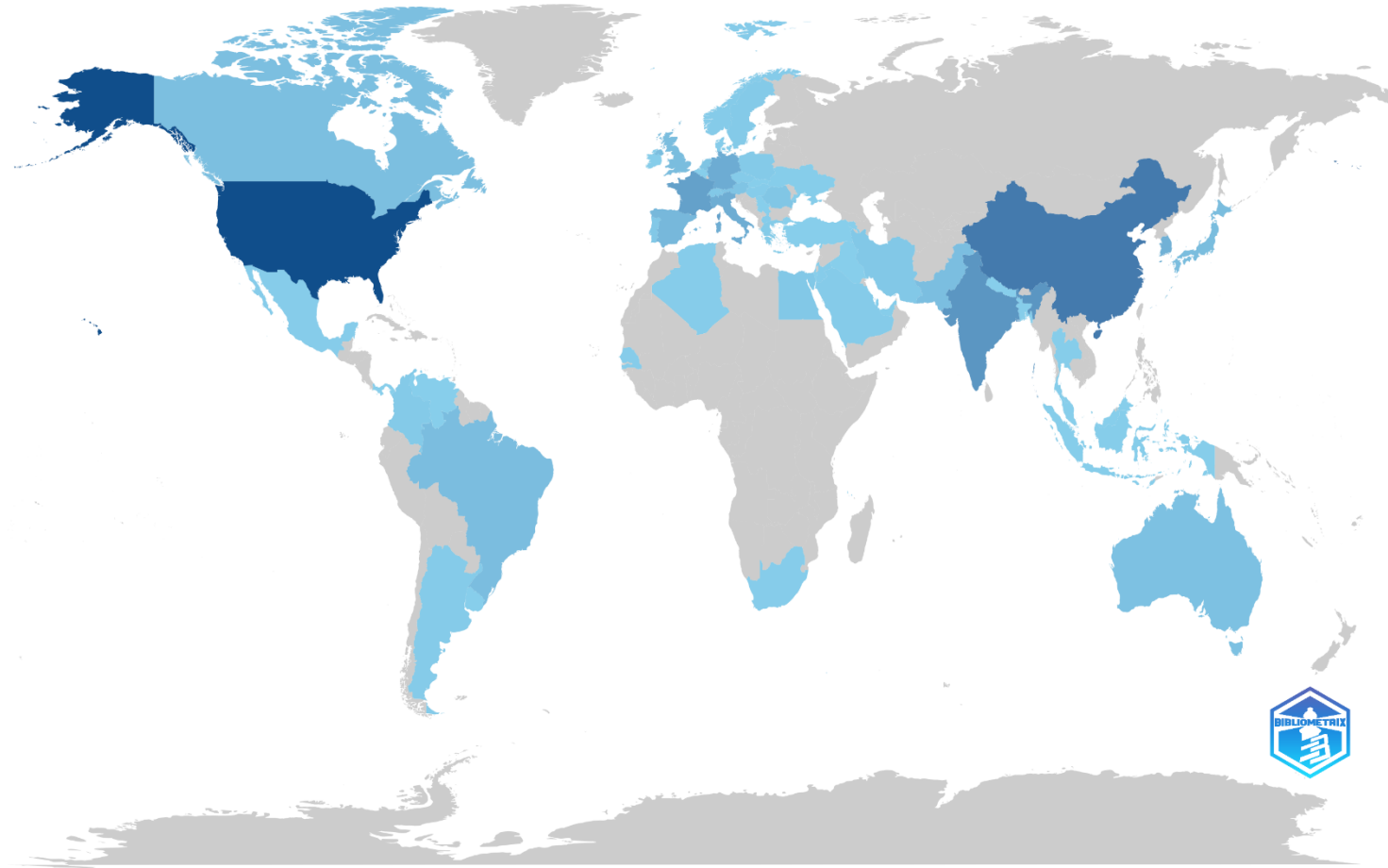
Plot Table

Excel

Country	Freq
Filter...	Filter...
USA	159
CHINA	101
INDIA	67
FRANCE	54
GERMANY	40
ITALY	39
SOUTH KOREA	34
SPAIN	26
JAPAN	21
UK	21

国家科学生产

Country's Scientific Production

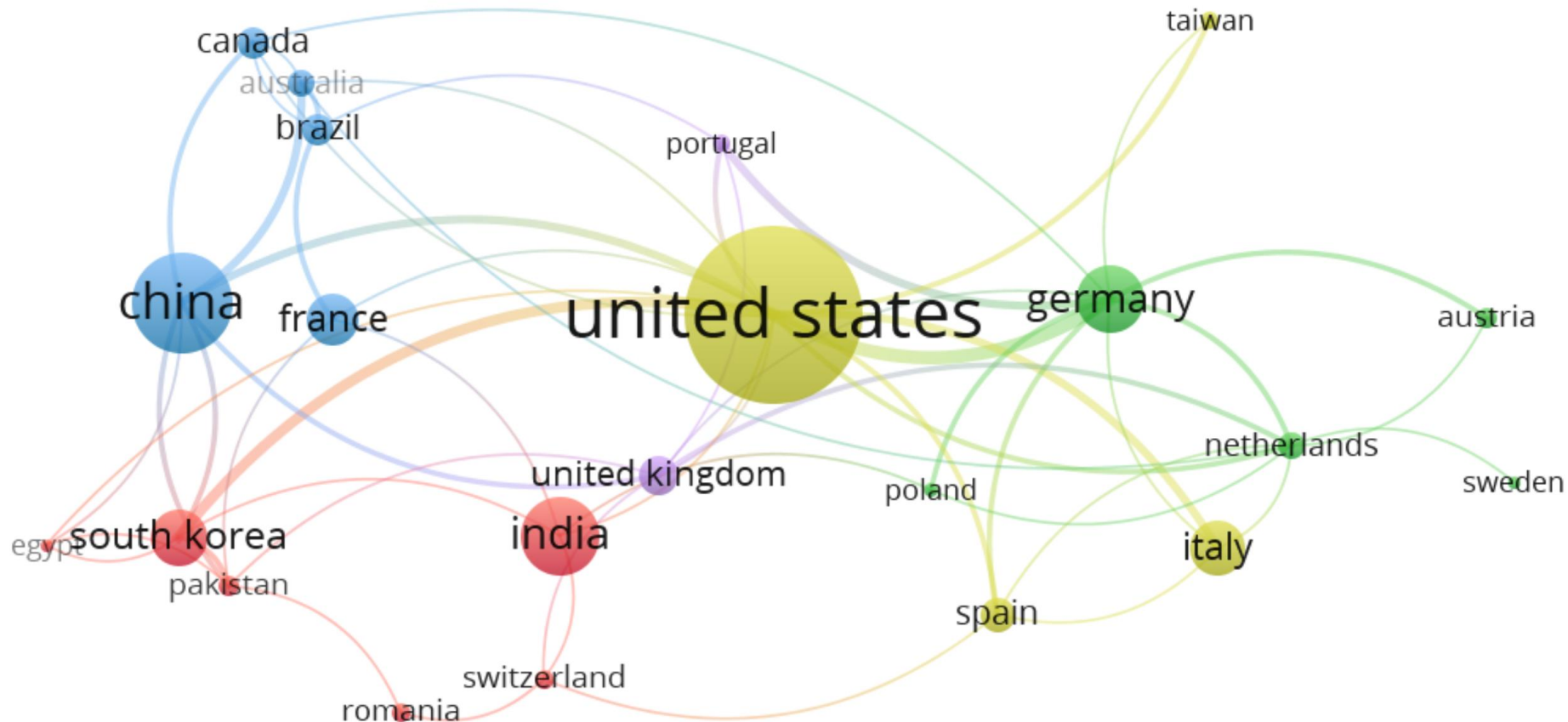


Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

北京工业大学 分布式系统的原理
Beijing University of Technology, Principles of Distributed Systems

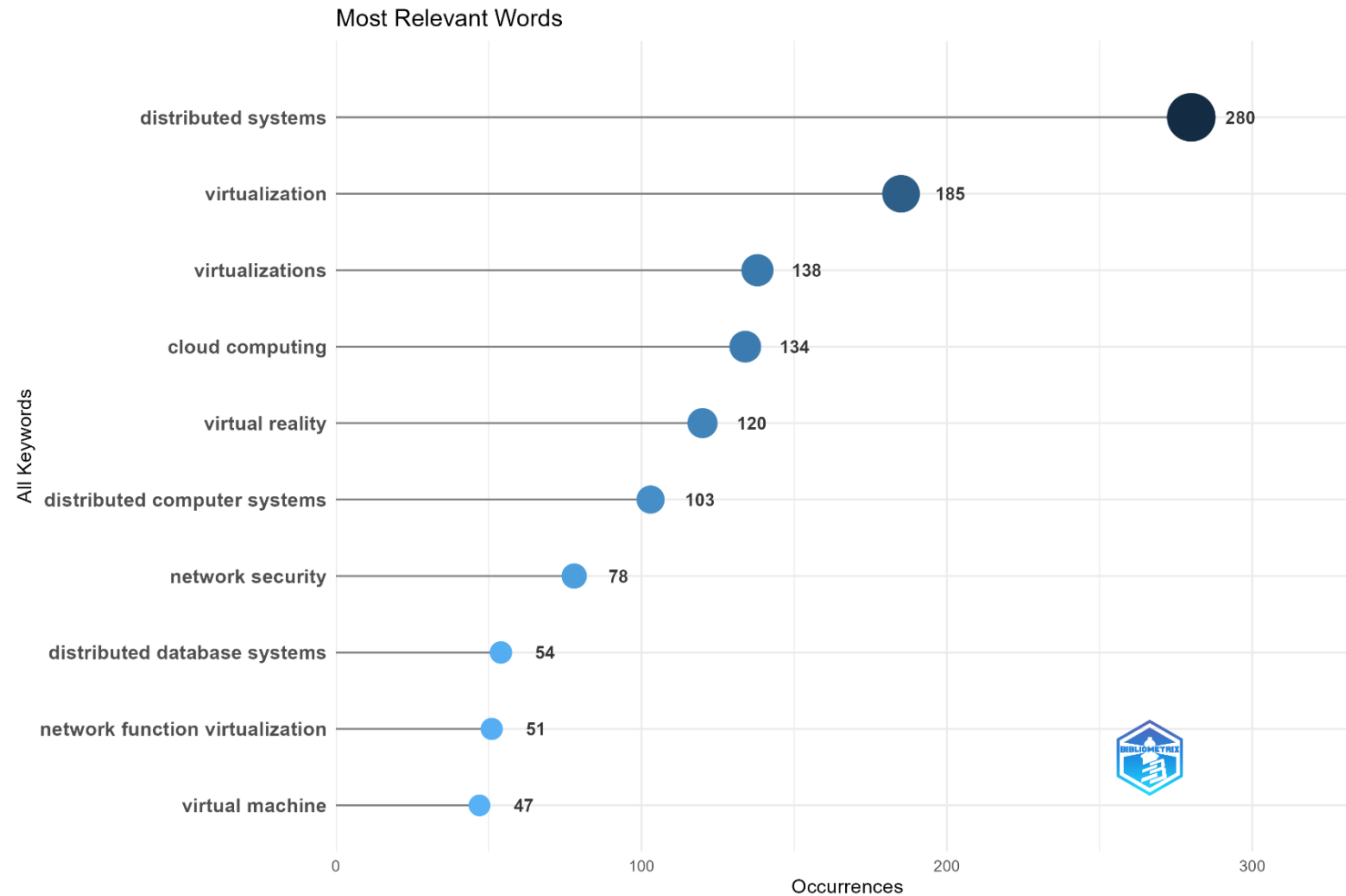
国家合作

Countries Collaborations



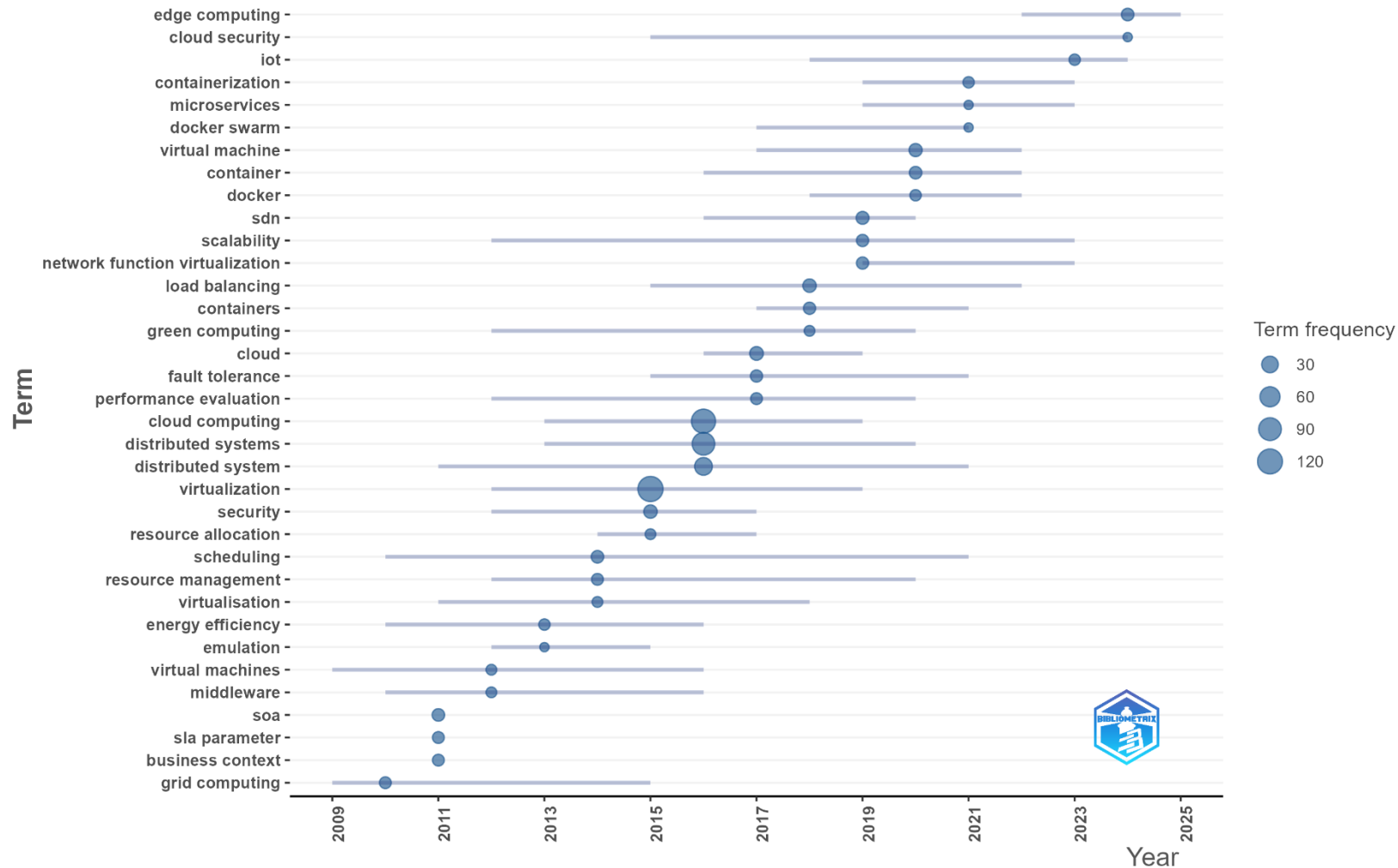
最相关的词汇

Most Relevant Words



Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

趋势话 Trend Topics



Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

主题地图 Thematic Map

The Four Quadrants

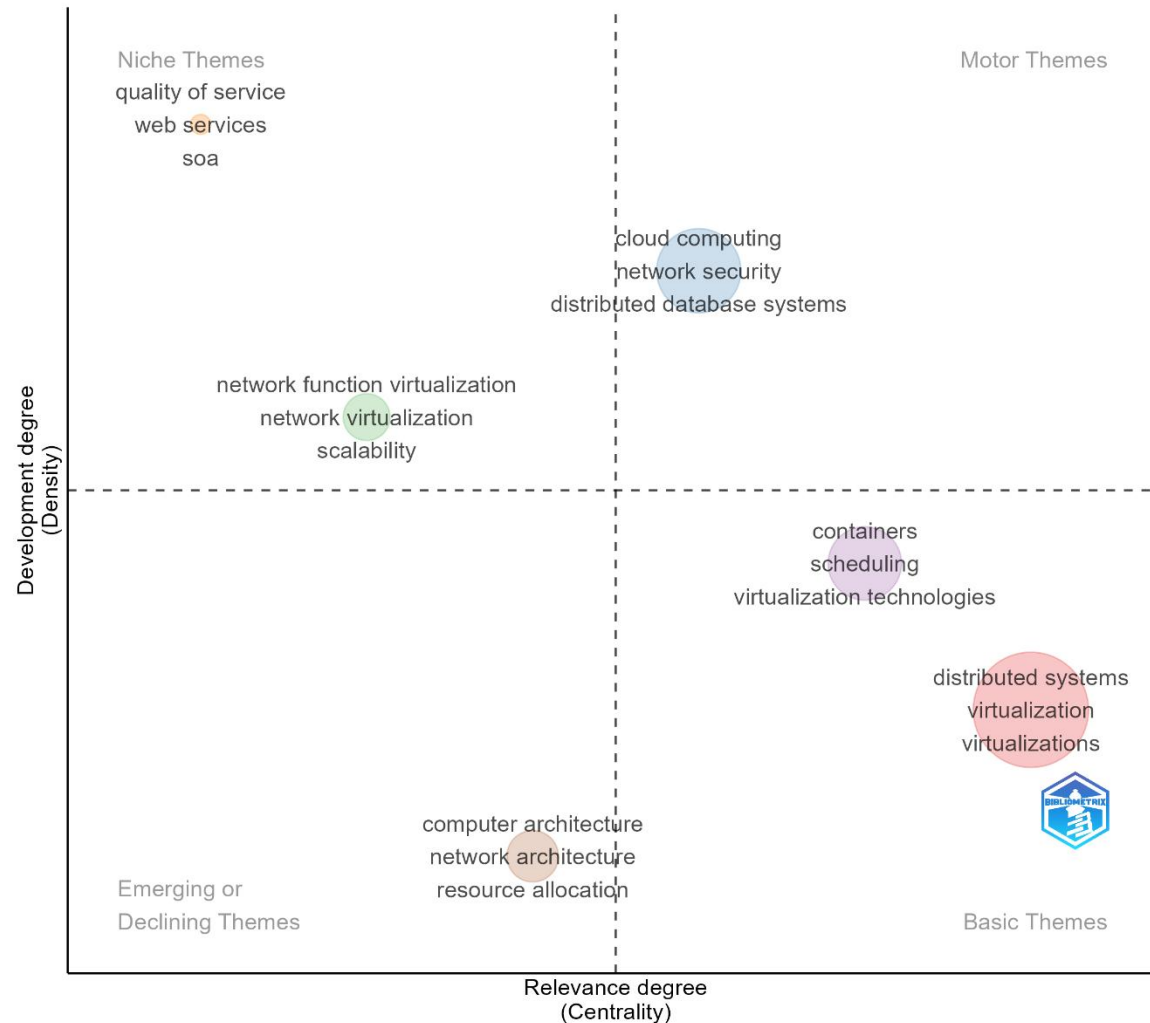
These two dimensions define four quadrants, each with a distinct strategic interpretation:

Upper-right (Motor themes): High centrality + High density. These are well-developed themes that are central to the field.

Lower-right (Basic/Transversal themes): High centrality + Low density. These themes are important to the field but not yet well-developed.

Upper-left (Niche themes): Low centrality + High density. These are well-developed but peripheral themes.

Lower-left (Emerging/Declining themes): Low centrality + Low density. These themes are both peripheral and underdeveloped.



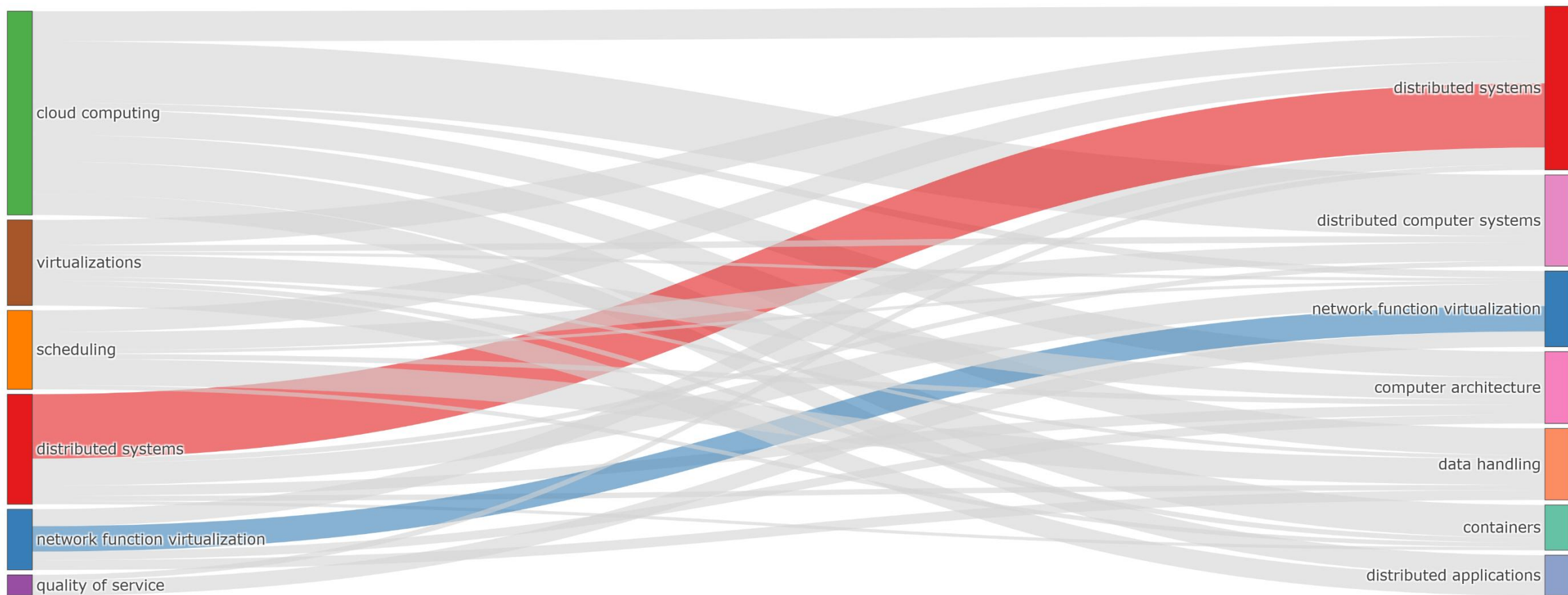
Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

项目讨论

Thematic Evolution

2007-2016

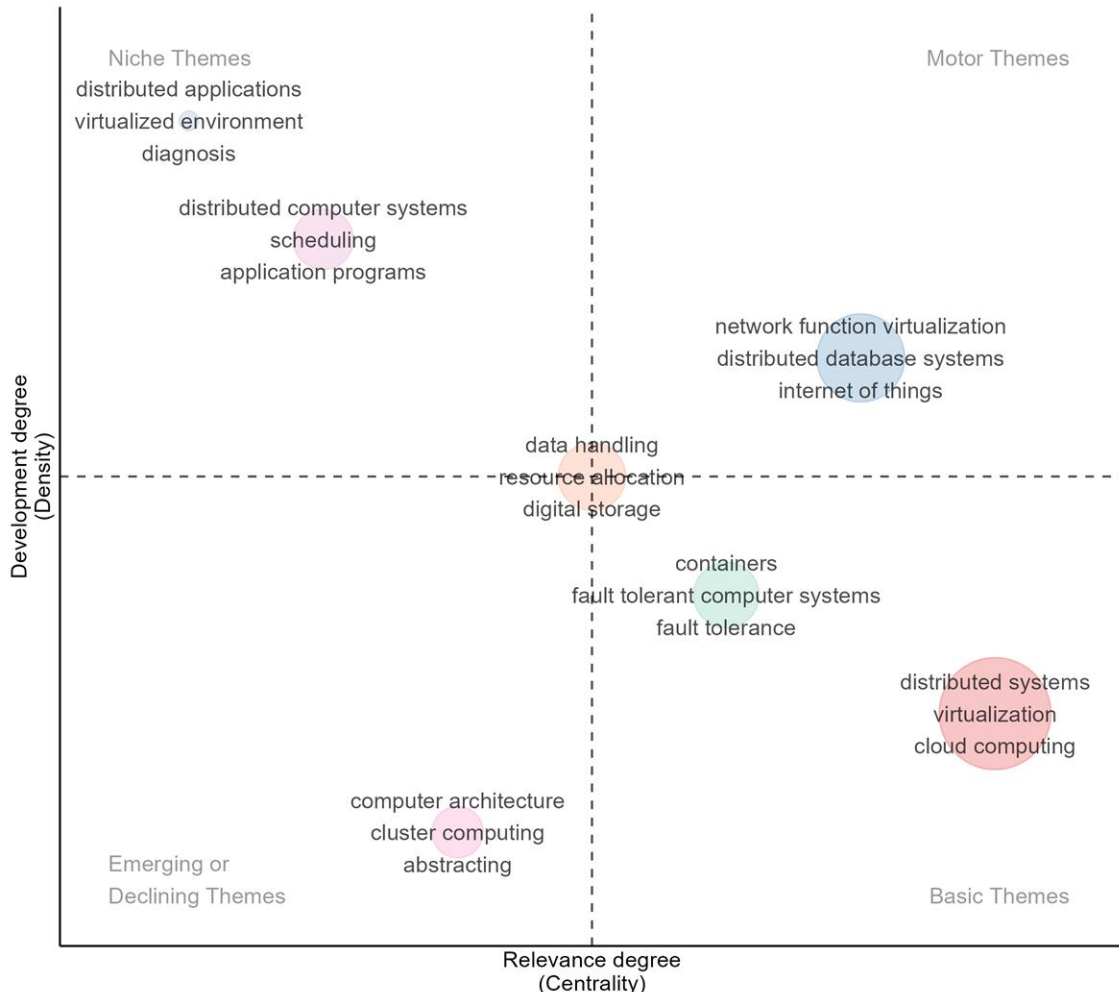
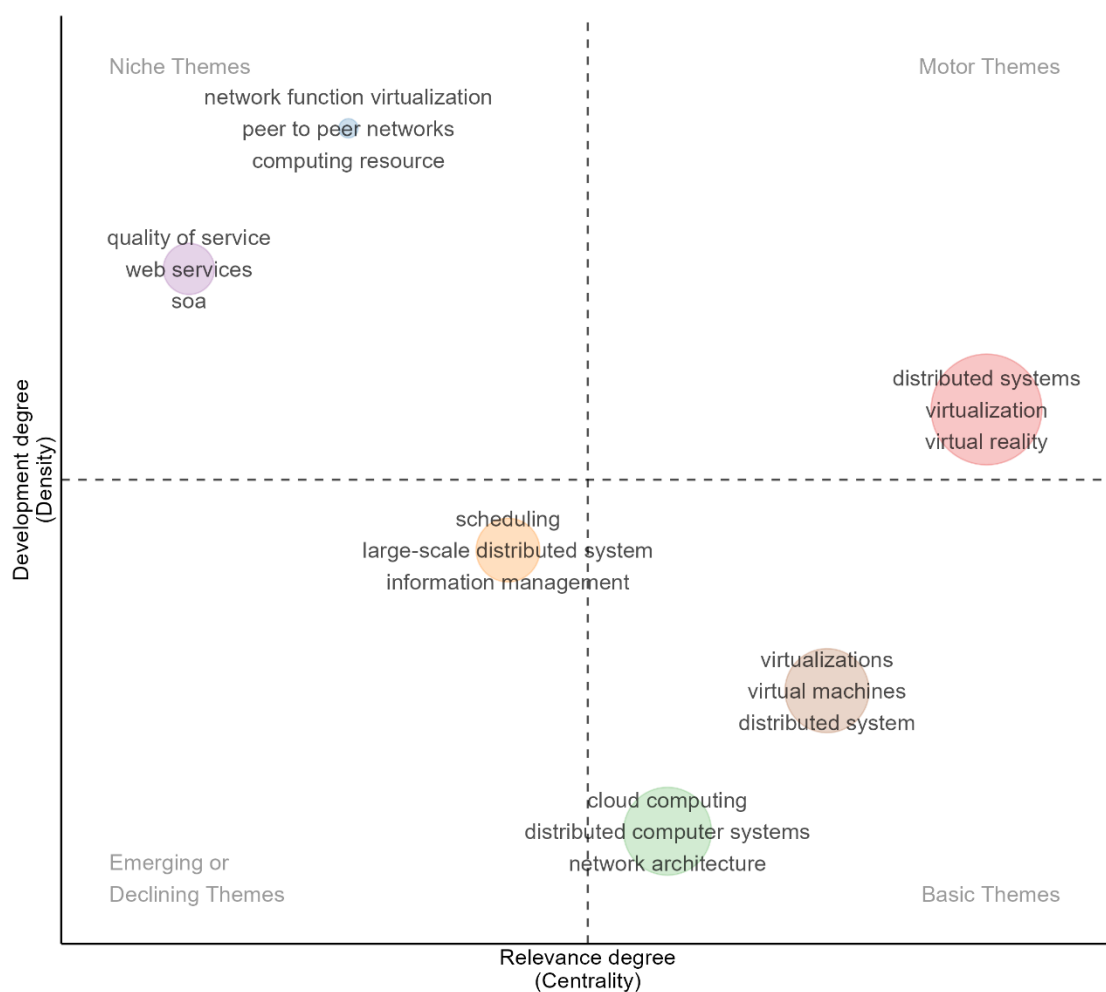
2017-2026



Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

项目讨论

Thematic Evolution



Week 3: Distributed System Processes and Threads [第3周：分布式系统进程与线]

项目讨论

Perform Bibliometric Analysis using BibShiny and VOSviewer

Task: Perform bibliometric analysis on the selected/registered topics before 8th May 2026 using BibShiny and VOSviewer.

Steps for Installation and Setting up Tools

1. Install BibShiny

Step 1: Install R/RStudio (if not installed already): Download from: [R project](#)

Step 2: Install BibShiny R package: Open RStudio and run this command:

```
install.packages("bibliometrix")
```

Step 3: Start BibShiny: Open RStudio and run:

```
library(bibliometrix)  
biblioshiny()
```

This will open BibShiny in your browser.

2. Install VOSviewer

Step 1: Download [VOSviewer](#):

Go to the VOSviewer website and download the latest version for your system.

Step 2: Install VOSviewer following the instructions provided on the website.

Step 3: Launch VOSviewer: Open the program after installation.

项目讨论

Perform Bibliometric Analysis using BibShiny and VOSviewer

Task Details

1. Select Keywords from your registered topic.
2. Use Scopus to collect bibliometric data (publication information, etc.).
3. Import the data file (e.g., CSV, BibTeX) into Bibshiny.
4. Create visualizations:
 1. Main overview, Life Cycle
 2. Main sources, countries, research themes, etc.
 3. Analyze and interpret the results from Bibshiny.
5. Import the data file (e.g., CSV, BibTeX) into VOSviewer.
6. Create visualizations:
 1. Co-authorship networks
 2. Keyword co-occurrence maps
 3. Analyze and interpret the results from VOSviewer.

项目讨论

Perform Bibliometric Analysis using BibShiny and VOSviewer

Task Submission: (before next lecture i.e. 18:00, 8th May 2026)

1. Update your project IDs (Latex project rename).
2. Refine your project titles/abstracts accordingly (Survey/Technical Advances/Review etc.).
3. Add analyses/figures/tables into your latex projects into relevant sections and discuss these results/analyses.

Distributed System Processes and Threads

What is a Process?

Process = A running program

It has its own memory space and operates independently.

What is a Thread?

Thread = A unit of execution within a process.

Threads share memory within the process, making them lightweight.

Key Benefits of Multithreading and Virtualization

Multithreading:

Handles many clients at once

Increases server responsiveness and scalability

Virtualization:

Helps simulate distributed systems

Allows resource isolation and efficient resource use